

1.

Vivado alat za projektovanje i verifikaciju FPGA baziranih sistema

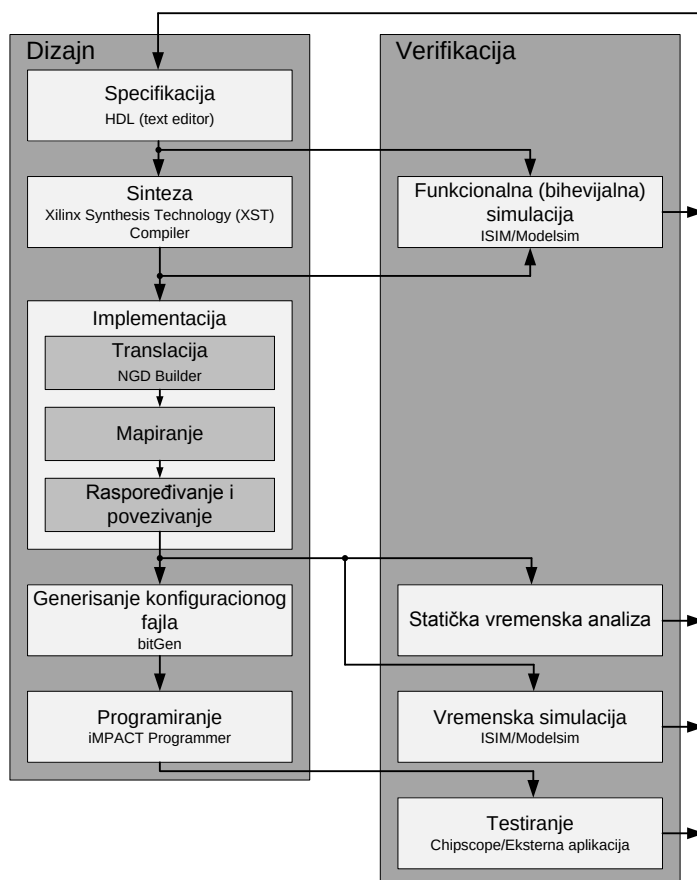
Savremeni digitalni elektronski sistemi odlikuju se izuzetno velikom složenosti. Savremena digitalna integrisana kola sastoje se od nekoliko milijardi individualnih tranzistora. Kako se složenost digitalnih sistema povećava, zadatak njihovog projektovanja postaje sve teži i teži. Pored toga, pojavljuje se još jedan problem, verifikacije korektnosti implementirane funkcionalnosti. Kako je tempo razvoja tehnologije fabrikacije integrisanih kola takav da se broj tranzistora koji se mogu smestiti na jedinicu površine integrisanog kola udvostručava svakih 18-24 meseca, dizajn i verifikacioni inženjeri suočavaju se sa sve većim raskorakom između njihove produktivnosti i rastućeg broja tranzistora koji im stoje na raspolaganju prilikom projektovanja novih sistema. Najbolji način za prevazilaženje ovog problema jeste dizajn i verifikacija digitalnih sistema na sve višim nivoima apstrakcije i progresivno uvođenje softverskih alata koji će automatski generisati korektnu fizičku implementaciju na osnovu modela visokog stepena apstrakcije.

Centralno mesto u povećanju produktivnosti projektovanja i verifikacije digitalnih elektronskih sistema zauzimaju jezici za opis hardvera, HDL (*Hardware Description Language*). HDL jezici su zamenili električne šeme kao sredstvo za opis digitalnog elektronskog sistema. Korišćenje HDL jezika omogućilo je modelovanje ponašanja digitalnog elektronskog sistema na različitim nivoima apstrakcije, u zavisnosti od trenutne potrebne prilike projektovanja digitalnog sistema. HDL jezici su takođe omogućili sistematski i formalizovan pristup problemu verifikacije digitalnih elektronskih sistema. Danas se u elektronskoj industriji koriste dva HDL jezika, VHDL i Verilog. Oba jezika su podjednaki ekspresivni mogućnosti i omogućavaju

dokumentovanje, modelovanje za potrebe simulacije i sinteze, kao i verifikaciju digitalnih elektronskih sistema.

1.1. Tipičan tok dizajna i verifikacije savremenog digitalnog sistema

Savremeni tok razvoja digitalnih elektronskih sistema, baziran na korištenju HDL jezika, prikazan je na slici 1.1, u slučaju implementacije korišćenjem FPGA (*Field Programmable Gate Array*) tehnologije. Tok razvoja digitalnih sistema baziran na korišćenju ASIC (*Application Specific Integrated Circuit*) tehnologije je gotovo identičan sa tokom prikazanim na slici 1.1, uz uključivanje nekoliko dodatnih koraka, i neće biti razmatran u ovoj knjizi.



Slika 1.1. Tok dizajna digitalnog elektronskog sistema baziran na korišćenju FPGA tehnologije

Prvi korak prilikom razvoja digitalnog elektronskog sistema je njegova **specifikacija**. Za ovu potrebu se koristi neki od raspoloživih HDL jezika. Pomoću HDL jezika razvija se model ponašanja i/ili strukture digitalnog elektronskog sistema.

Sledeći korak prilikom razvoja digitalnog elektronskog sistema jeste korak **sinteze**. Pre samog koraka sinteze, alat za sintezu vrši sintaksnu analizu HDL modela kao i analizu hijerarhijske strukture razvijenog HDL modela. U procesu sinteze vrši se transformacija HDL modela dizajna sa visokog nivoa apstrakcije na niži nivo, koji je pogodan za sledeće korake u procesu implementacije. Ova reprezentacija naziva se netlista (*netlist*) i predstavlja model digitalnog sistema opisan pomoću elementarnih logičkih kapija, flip-flopova i lečeva. Netlista zapravo predstavlja pandam nekadašnjoj električnoj šemi ali se ona automatski generiše od strane alata za sintezu.

Nakon koraka sinteze prelazi se na fazu implementacije automatski generisane netliste na odabranu hardversku platformu. U prikazanom toku, hardverska platforma je neko od raspoloživih FPGA kola. Faza implementacije dizajna sastoji se iz tri koraka:

1. **Translacija** – u ovom koraku vrši se objedinjavanje svih ulaznih netlisti (ukoliko ih ima više od jedne) kako bi se dobila jedna, objedinjena netlista.
2. **Mapiranje** – u koraku mapiranja objedinjena netlista, koja se sastoji od elementarnih logičkih kapija, flip-flopova i lečeva, mapira se na hardverske resurse koji stoje na raspolaganju u odabranoj FPGA komponenti. U slučaju FPGA komponenti, pod hardverskim resursima podrazumevamo memorije malog kapaciteta (16-64 bita) za realizaciju Bulovih funkcija (takozvane *lookup* tabele, LUT), konfigurabilne flip-flopove/lečeve, memorije većeg kapaciteta (18-36 kbita) za realizaciju memorijskih elemenata (takozvane *Block RAM* memorije), optimizovane hardverske blokove za izvođenje aritmetičkih operacija (takozvane DSP blokove), itd.
3. **Raspoređivanje i povezivanje** – u ovom koraku se mapiran dizajn fizički raspoređuje po FPGA komponenti. Svakom od korišćenih hardverskih resursa određuje se tačno mesto na FPGA čipu i, koristeći raspoložive kanale za rutiranje, njegovi ulazi i izlazi povezuju se sa asociranim hardverskim resursima.

Faza **generisanja konfiguracionog fajla** predstavlja poslednji korak u procesu razvoja digitalnog elektronskog sistema pomoću FPGA tehnologije. U ovom koraku, na osnovu raspoređenog i povezanog dizajna, generiše se binarni konfiguracioni fajl koji će biti iskorišćen za konfigurisanje FPGA komponente kako bi ona bila u stanju da implementira projektovani digitalni elektronski sistem. Konfiguracioni fajl poznat je i pod imenom *bitstream* fajl.

Nakon što je kreiran konfiguracioni fajl, može se pristupiti fazi **programiranja** FPGA komponente. U ovom koraku se generisani konfiguracioni fajl upisuje u FPGA komponentu, koja se na osnovu njega interno konfiguriše, i na taj način „transformiše“ u projektovani digitalni elektronski sistem.

Uporedo sa procesom implementacije dizajna teče i proces njegove **verifikacije**. Verifikacija dizajna ima za cilj da proveriti da li model digitalnog elektronskog sistema koji se implementira zadovoljava željene funkcionalne i vremenske karakteristike. U praksi se verifikacija izvodi korišćenjem ekstenzivnih simulacija ponašanja razvijenog HDL modela digitalnog elektronskog sistema koji se implementira. Verifikacija dizajna obuhvata sledeće korake:

1. **Funkcionalnu (bihevioralnu) simulaciju** – ovom simulacijom omogućava se verifikacija sintaksne ispravnosti razvijenog HDL modela kao i implementirane funkcionalnosti. Ova vrsta simulacije koristi se u ranoj fazi razvoja digitalnog sistema, pre njegove sinteze, i omogućava projektantu da ispravi uočene greške i napravi neophodne izmene na dizajnu rano u procesu projektovanja. Ovo je od izuzetnog značaja, zato što su koraci sinteze i implementacije računski vrlo zahtevni i mogu trajati i nekoliko desetina časova, tako da je otklanjanje grešaka nakon završene faze sinteze i implementacije vrlo skupo. Ono što se međutim ne može verifikovati pomoću funkcionalne simulacije, jeste vremensko ponašanje projektovanog digitalnog sistema (jer nam vremenske karakteristike sistema još uvek nisu poznate jer sistem još uvek nije fizički implementiran, to će se desiti tek nakon faze sinteze i implementacije). Ovo je i razlog zašto se ova vrsta simulacije naziva i bihevioralna, jer nam daje odgovore samo na pitanje da li je ponašanje/funkcionalnost projektovanog sistema u skladu se željenim.
2. **Vremensku (tajming) simulaciju** – ova vrsta simulacije koristi informacije o realnim kašnjenjima signala, koja su posledica prolaska signala kroz konkretne hardverske resurse (LUT memorije, flip-flopove, itd.) kao i linije za rutiranje signala unutar dizajna koji se projektuje, i omogućava proveru ne samo korektnog funkcionalnog ponašanja sistema već i da li on zadovoljava željene vremenske karakteristike. Ova vrsta simulacije koristi se nakon što je dizajn sintetizovan i implementiran, jer je nakon ovih koraka poznato kašnjenje svakog od unutrašnjih signala prisutnih u finalnom dizajnu. Proces estimacije/ekstrakcije realnih kašnjenja unutrašnjih signala izvodi se automatski, korišćenjem odgovarajućih alata i modela, a estimirana kašnjenja se ugrađuju u simulacioni model implementiranog dizajna. Bitno je naglasiti da je i ovaj model napisan korišćenjem nekog od dva HDL jezika, ali ovaj put automatski, od strane odgovarajućeg alata. Vremenska simulacija je izuzetno računski zahtevna, i po pravilu zahteva 10 do 100 puta duže vreme simulacije od odgovarajuće funkcionalne simulacije.
3. Ukoliko se digitalni sistem projektuje koristeći metodologiju sinhronog sekvencijalnog dizajna, koja je predmet ovog kursa, tada se vremenska simulacija može zameniti procesom **statičke vremenske analize** (*Static Timing Analysis*). Statička vremenska analiza na analitički i formalan način izračunava kašnjenja signala duž svakog od mogućih unutrašnjih puteva u dizajnu i proverava da li kritične putanje (putanje sa najvećim kašnjenjem) zadovoljavaju postavljena vremenska ograničenja, odnosno da li implementirani dizajn može da radi na zahtevanoj učestanosti globalnog

sinhronizacionog (klok, *clock*) signala. Statička vremenska analiza je vrlo brza i zahteva vrlo malo računskog vremena.

Poslednji korak u procesu verifikacije dizajna predstavlja njegovo **testiranje** na konkretnoj FPGA platformi. Proces hardverskog testiranja je vrlo zahtevan u pogledu vremena potrebnog za njegovo kompletiranje, prvenstveno zbog ograničenih pristupnih tačaka za stimulisanje i observaciju ponašanja projektovanog sistema. Takođe, ispravljanje uočenih grešaka u dizajnu je vrlo skupo, jer zahteva ponovni proces sinteze i implementacije. Zbog toga se u praksi ovaj korak nastoji izbeći ili barem minimizovati. Ukoliko se proces funkcionalne verifikacije i statička vremenska analiza pravilno sprovedu, proces hardverskog testiranja postaje praktično suvišan.

Dodatni korak, koji postaje sve više interesantan u procesu razvoja savremenih digitalnih sistema, jeste korak **estimacije potrošnje električne energije** razvijenog sistema. Pored toga što projektovani sistem mora da poseduje željenu funkcionalnost, da ima željene vremenske karakteristike (željenu brzinu rada), sve češće je neophodno da projektovani sistem ima i potrošnju koja je manja od maksimalno dozvoljene. Ovo je pogotovo važno ukoliko će se projektovani sistem koristiti u mobilnim embeded aplikacijama, koje trenutno dominiraju tržištem digitalne elektronike. Korišćenjem odgovarajućih alata moguće je, nakon implementacije dizajna, estimirati, sa većom ili manjom preciznošću, očekivanu potrošnju električne energije implementiranog digitalnog sistema.

U praksi se za sve navedene korake koriste posebno razvijeni softverski alati. Ovi alati mogu biti samostalni, ali najčešće dolaze u obliku jednog, integrisanog razvojnog okruženja (IDE, *Integrated Development Environment*) za projektovanje i verifikaciju digitalnih elektronskih kola. Najveći proizvođači ovih softverskih alata su kompanije Cadence, Synopsys i MentorGraphic, koje su se specijalizovale za razvoj alata u oblasti ASIC dizajna. U oblasti FPGA dizajna, najpoznatiji proizvođači softverskih alata su kompanije Xilinx i Altera. U nastavku ove glave detaljno će biti prikazano korišćenje softverskog paketa Vivado Design Suite kompanije Xilinx, pomoću kojega je moguće sprovesti sve gorenavedene korake prilikom projektovanja i verifikacije savremenog digitalnog elektronskog sistema.

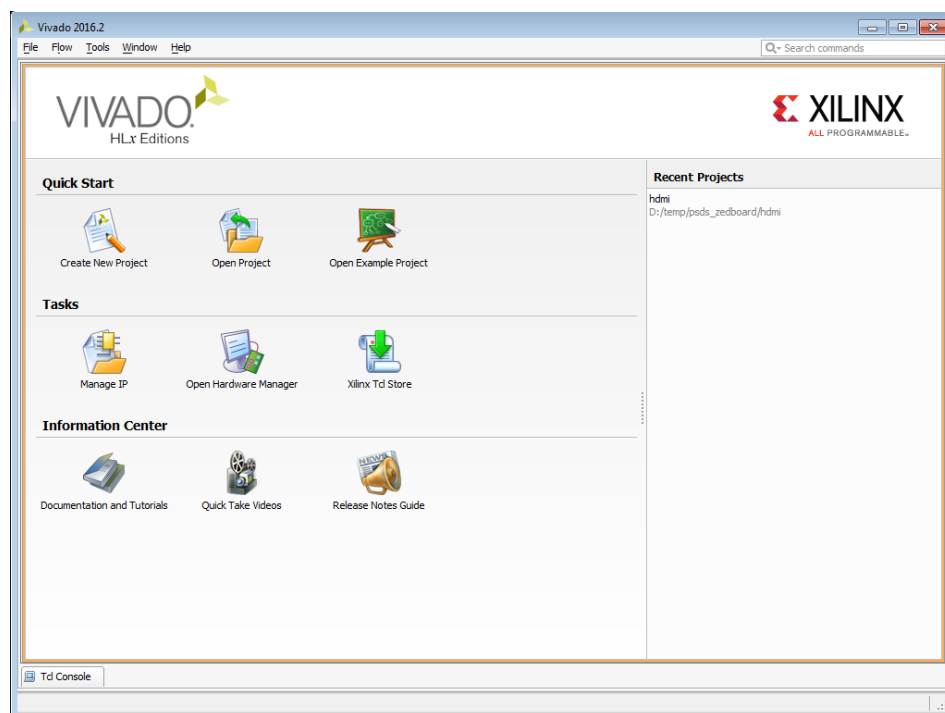
1.2. Vivado alat

Alat Vivado Design Suite predstavlja integrisano razvojno okruženje namenjeno implementaciji digitalnih elektronskih sistema pomoću FPGA komponenti kompanije Xilinx. Ovo razvojno okruženje zapravo predstavlja čitav skup odvojenih alata, kojima se može pristupiti preko jednog, zajedničkog grafičkog okruženja. Korišćenjem Vivado Design Suite alata moguće je proći kroz sve korake prilikom dizajna i verifikacije digitalnog elektronskog sistema prikazane na slici 1.1.

U nastavku će biti prikazan način rada sa Vivado Design Suite razvojnim okruženjem na primeru najjednostavnijeg mogućeg digitalnog elektronskog sistema, koji se sastoji

samo iz jednog dvoulaznog I logičkog kola. Sve navedene opcije i mogućnosti alata odnose se na poslednju verziju Vivado Design Suite alata u trenutku pisanja ovog udžbenika, Vivado Design Suite 2016.2.

Nakon pokretanja Vivado 2016.2 alata, dvoklikom na ikonicu sa desktopa ili selektovanjem opcije Start → All Programs → Xilinx Design Tools → Vivado 2016.2 → Vivado 2016.2 iz Start menija, pojaviće se glavni prozor Vivado Design Suite alata, prikazan na slici 1.2.



Slika 1.2. Izgled glavnog prozora Vivado Design Suite alata

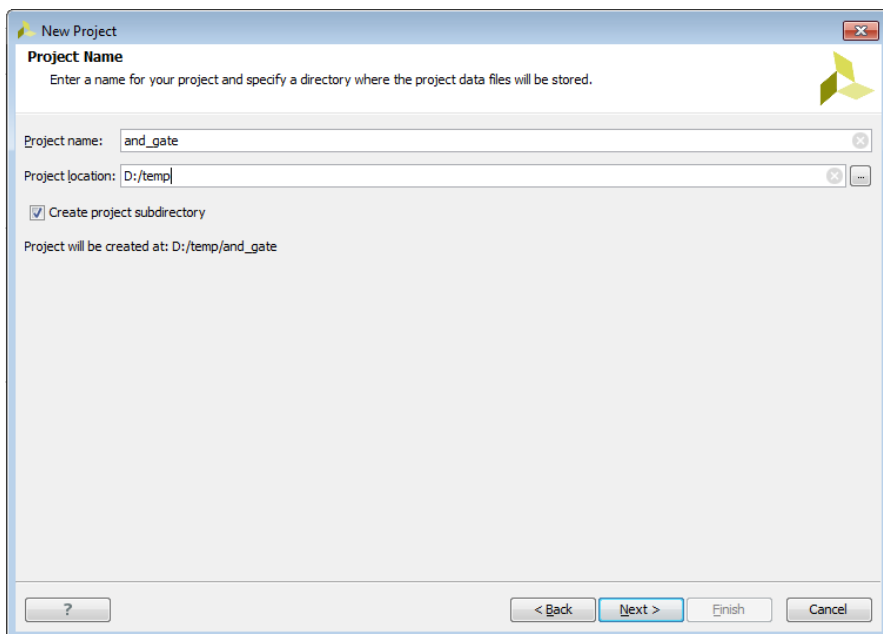
U ovom trenutku na raspolaganju nam stoji nekoliko mogućnosti:

- možemo kreirati novi projekat, pritiskom na dugme „Create New Project“ u gornjem levom uglu prozora
- možemo otvoriti neki projekat koji smo ranije kreirali, pritiskom na dugme „Open Project“
- možemo otvoriti neki od primera, u vidu kompletnih projekata, koji mogu biti vrlo zgodni za upoznavanje sa alatom, pritiskom na dugme „Open Example Project“

- možemo pretraživati bazu raspoloživih IP jezgara, gotovih modula koji implementiraju često korišćene funkcionalnosti, pritiskom na dugme „*Manage IP*“
- možemo ostvariti vezu sa FPGA komponentom koja je trenutno povezana sa PC računarom preko JTAG porta, pritiskom na dugme „*Open Hardware Manager*“, kako bismo izvršili njeno programiranje sa odabranim konfiguracionim fajlom ili pak izvršili hardversko testiranje korišćenjem ugrađenih logičkih analizatora
- možemo pretraživati bazu Tcl skripti, razvijenih od strane kompanije Xilinx i korisnika Vivado alata, pritiskom na dugme „*Xilinx Tcl Store*“
- možemo pretraživati bazu dokumenata koji detaljnije opisuju različite metodologije razvoja sistema baziranih na FPGA tehnologiji, kao i postojećih tutorijala, ili pristupiti kompletnoj dokumentaciji koju nudi kompanija Xilinx, pritiskom na dugme „*Documentation and Tutorials*“
- možemo pregledati kratke video materije koji objašnjavaju različite mogućnosti Vivado alata i pravilne načine na koje se one mogu koristiti, pritiskom na dugme „*Quick Take Videos*“
- možemo pristupiti informacijama o novim mogućnostima Vivado alata, koje su postale dostupne u poslednjoj verziji alata, kao i informacijama u vezi sa načinom instaliranja Vivado alata, pritiskom na dugme „*Release Notes Guide*“
- možemo odabrati neki od projekata na kojima smo nedavno radili, u okviru prozora „*Recent projects*“

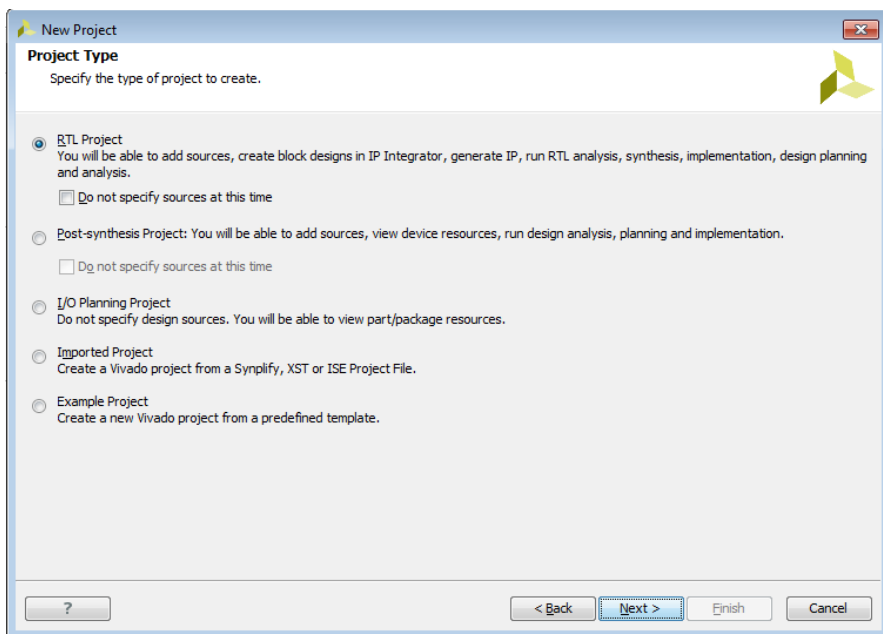
1.3. Kreiranje novog projekta

Pošto mi započinjemo rad na novom projektu odabraćemo opciju „*Create New Project*“ (isti efekat postiže se i ako selektujemo *File/New Project* opciju iz menija) kako bismo definisali karakteristike novog projekta. Nakon pritiska na dugme „*Create New Project*“ pojaviće se novi prozor, koji nas informiše o tome da smo pokrenuli čarobnjaka koji će nam pomoći prilikom kreiranja novog projekta. Pritiskom na dugme *Next* prelazimo na sledeći prozor, „*Project Name*“, u kojem zapravo započinjemo proces kreiranja novog projekta, prikazan na slici 1.3.



Slika 1.3. „New Project – Project Name“ prozor sa potrebnim podešavanjima

Prilikom kreiranja novog projekta prvi korak je da se definiše njegovo ime (polje „Project name“ sa slike 1.3), mesto na disku gde će biti smešteni svi projektni fajlovi (polje „Project location“) kao i da li je potrebno kreirati u pod-direktorijum u direktorijumu koji je specificiran kao mesto na kome će se čuvati projektni fajlovi („Create project subdirectory“). U našem slučaju ime projekta će biti *and_gate*, lokacija na disku može se izabrati proizvoljno, uz napomenu da putanja do odabrane lokacije **ne** sme sadržati *space* karaktere. Pritiskom na dugme *Next* prelazimo na sledeći prozor podešavanja projekta, prikazan na slici 1.4.

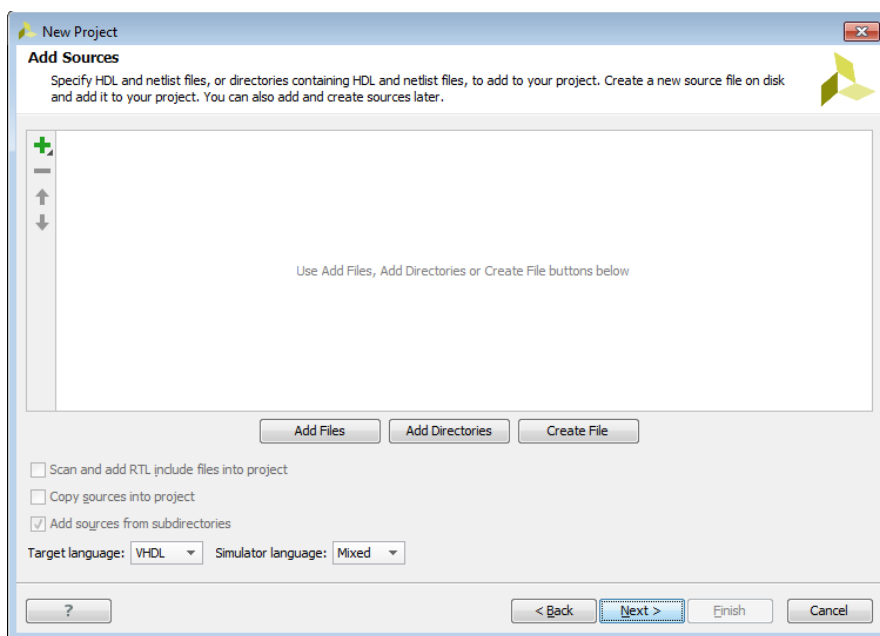


Slika 1.4. „New Project – Project Type“ prozor sa potrebnim podešavanjima

U okviru „Project Type“ prozora specificira se tip budućeg projekta. Korisnik može odabrati jednu od sledećih opcija:

- *RTL Project* – najčešće korišćeni tip projekta, u okiru kojega je moguće dodatavi izvorne fajlove, kreirati blok dijagrame, generisati i dodavati IP jezgra, izvršiti korake sinteze, implementacije, planiranja i analize dizajna
- *Post-synthesis Project* – ovaj tip projekta omogućava da se u prethodno sintetizovan projekat dodaju novi izvorni fajlovi, analiziraju potrebni resursi, i izvrši planiranje i implementacija projekta
- *I/O Planning Project* – u ovom tipu projekta nije moguće specificirati izvorne fajlove, niti moguće izvršiti sintezu i implementaciju. U ovom slučaju moguće je samo sprovesti planiranje načina na koji će interfejs signali budućeg digitalnog sistema biti raspoređeni po odabranoj FPGA komponenti. Ovaj tip projekta koristi se u ranoj fazi projekta, kada je potrebno pravilno odabrati ciljnu FPGA komponentu pomoću koje će na kraju biti realizovan čitav digitalni sistem.
- *Imported Project* – omogućava automatsko generisanje Vivado projekta na osnovu projekta koji je bio razvijen pomoću drugih ISE alata, kao što su Synosys Synplify ili Xilinx XST ili Foundation ISE alati
- *Example Project* – omogućava kreiranje Vivado projekta, korišćenjem nekog od unapred definisanih šablona

U našem slučaju potrebno je odabrati prvi tip projekta, *RTL Project*, koji se i inače najčešće koristi. Takođe je potrebno obezbediti da opcija „*Do not specify sources at this time*“ ostane neaktivna, jer želimo da u nastavku definišemo i neke od izvornih fajlova projekta pomoću čarobnjaka. Pritiskom na dugme *Next*, dolazimo do „*Add Sources*“ prozora, prikazanog na slici 1.5.



Slika 1.5. „*New Project – Add Sources*“ prozor

„*Add Sources*“ prozor omogućava nam da u kreirani projekat uključimo izvorne dizajn fajlove. Ovo su zapravo HDL fajlovi u okviru kojih se nalazi model digitalnog sistema koji želimo da realizujemo. Moguće je:

- uključiti prethodno napisane HDL modele, pritiskom na dugme „*Add Files*“,
- čitave direktorijume u kojima se nalaze prethodno razvijeni HDL modeli, pritiskom na dugme „*Add Directories*“ ili
- kreirati potpuno nov HDL model od početka, pritiskom na dugme „*Create File*“.

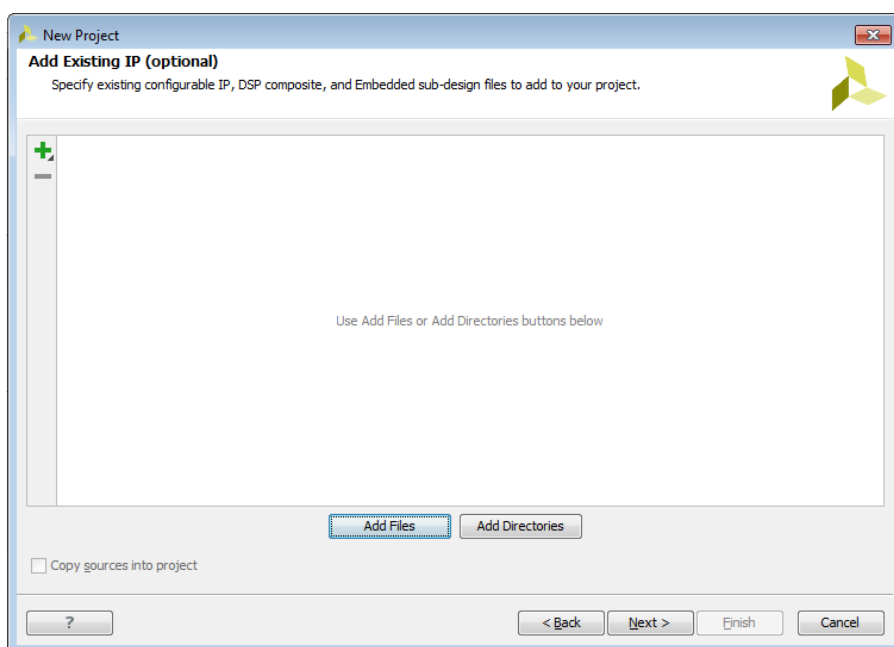
Pored ovih opcija, moguće je izabrati HDL jezik koji će biti korišćen od strane Vivado alata prilikom automatskog kreiranja izvornih fajlova (na primer fajlova koji će biti kreirani prilikom generisanja odabranih IP jezgara ili prilikom generisanja simulacionih modela, itd.), pomoću opcije „*Target language*“. Takođe je moguće izabrati i tip

ugrađenog simulatora koji će biti korišćen, pomoću opcije „*Simulator language*“. U našem slučaju ove opcije treba da budu podešene na vrednosti *VHDL* i *Mixed* respektivno.

Dodavanje HDL izvorišnih fajlova u projekat moguće je izvršiti i kasnije, nakon što se projekat kreira. U našem primeru mi ćemo koristiti ovaj pristup, pa pritiskom na dugme *Next* prelazimo na „*Add Existing IP*“ prozor, prikazan na slici 1.6, u okviru kojega je moguće specificirati postojeća IP jezgra koja želimo da koristimo u okviru projekta.

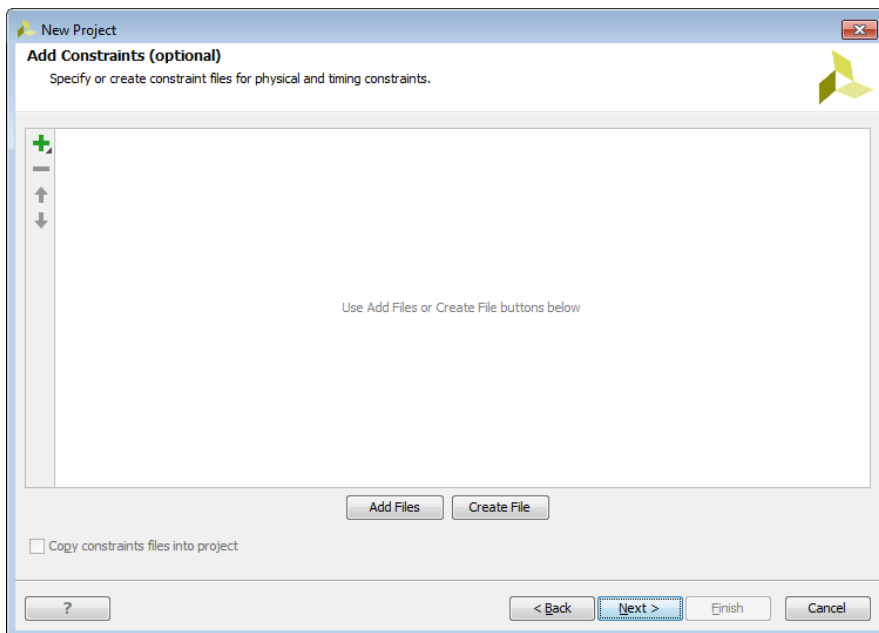
Pomoću dugmeta „*Add Files*“, moguće je dodati definicione fajlove za IP jezgra koja želimo da uključimo u projekat. Ovi definicioni fajlovi moraju da budu u jednom od formata koji prepoznaje Xilinx Vivado Design Suite alat (.xco, .xci, .xdx, .mdl, .sdx, .bd).

Pomoću dugmeta „*Add Directories*“ moguće je specificirati direktorijum u kome se nalazi skup definicionih fajlova za IP jezgra koja želimo da uključimo u projekat.



Slika 1.6. „*New Project – Add Existing IP*“ prozor

U našem primeru nemamo ni jedno IP jezgro koje želimo da uključimo u projekat, pa pritiskom na dugme *Next* možemo preći na sledeći prozor, „*Add Constraints*“ prikazan na slici 1.7.



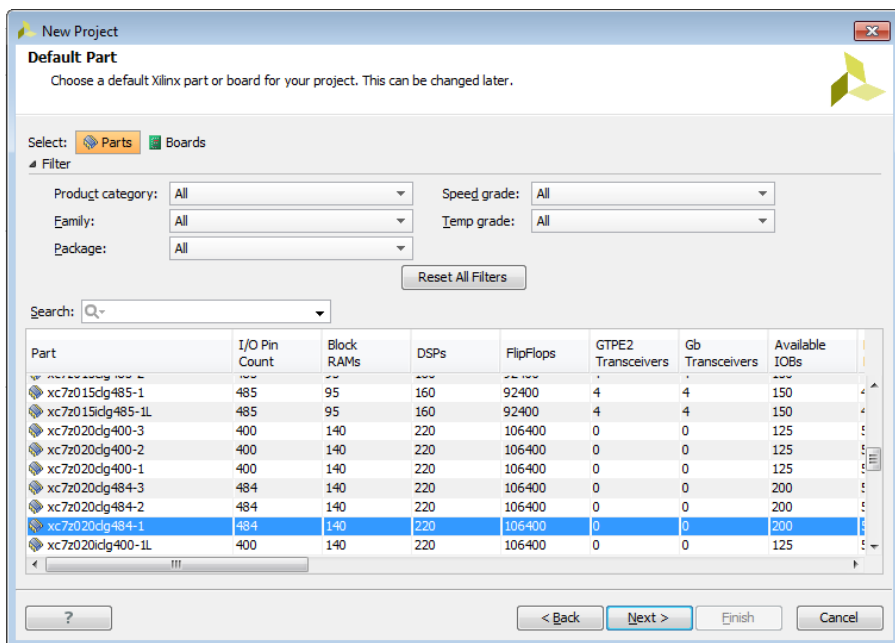
Slika 1.7. „New Project – Add Constraints“ prozor

U okviru „Add Constraints“ prozora moguće je uključiti postojeći, ili kreirati novi, fajl koji sadrži lokacijska i vremenska ograničenja koja je neophodno ispoštovati prilikom implementacije dizajna.

Pritiskom na dugme „Add Files“ moguće je u projekat uključiti prethodno kreirani fajl sa ograničenjima, dok je pritiskom na dugme „Create File“ moguće kreirati novi fajl sa ograničenjima.

Kao i u slučaju dodavanja HDL izvornih fajlova, fajlove sa ograničenjima moguće je uključiti u projekat i naknadno, kada se on kreira.

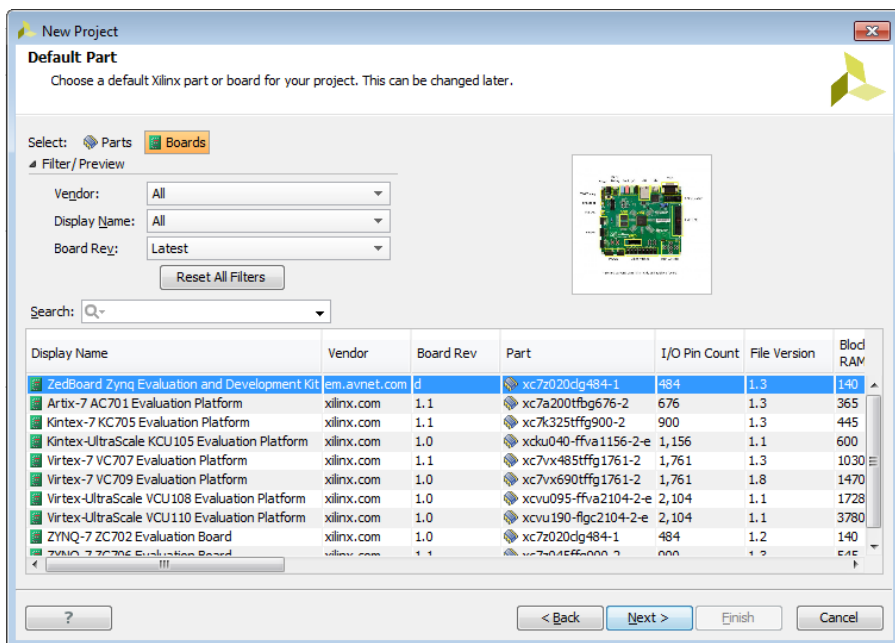
Pritiskom na dugme *Next*, prelazimo na „Default Part“ prozor, prikazan na slici 1.8. U okviru ovog prozora specificira se FPGA komponenta, ili FPGA razvojni sistem, na kojoj će biti implementiran digitalni sistem koji se projektuje.



Slika 1.8. „New Project – Default Part“ prozor, Parts kartica

U okviru ovog prozora moguće je izabrati FPGA komponentu (korišćenjem *Parts* kartice) koju želimo da koristimo prilikom fizičke implementacije dizajna. Sve raspoložive FPGA komponente kompanije Xilinx, koje podržava Vivado alat, nalaze se izlistane u okviru donjeg prozora sa slike 1.8. Radi lakšeg pronalaženja željene FPGA komponente mogu se koristiti neka od *Filter* polja („*Product category*“, *Family*, *Package*, „*Speed grade*“ i „*Temp grade*“).

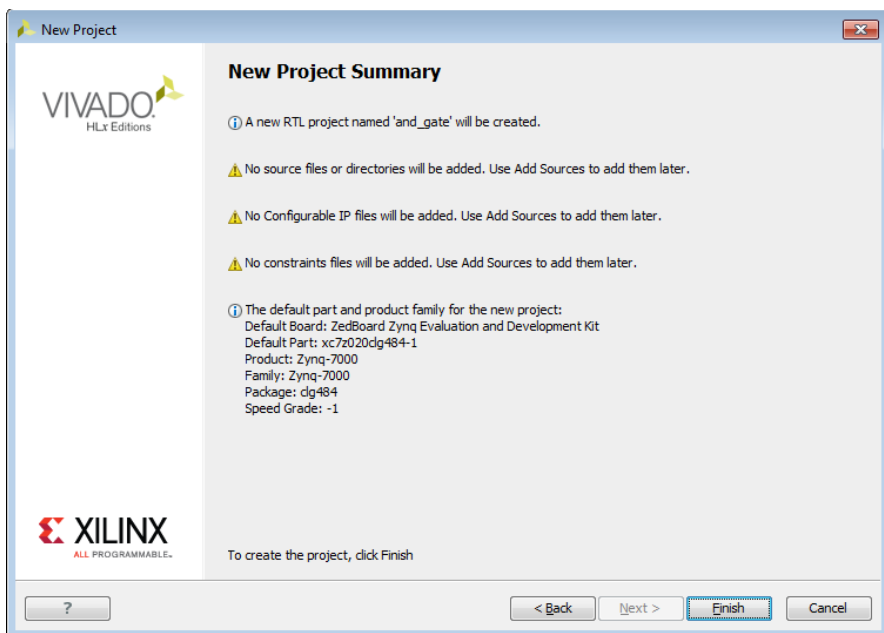
Ukoliko imamo neki od standardnih Xilinx FPGA razvojnih sistema koji želimo da koristimo prilikom implementacije dizajna, možemo ga jednostavno odabrati iz liste ponuđenih razvojnih sistema, pritiskom na *Boards* karticu, kao što je prikazano na slici 1.9.



Slika 1.9. „New Project – Default Part“ prozor, Boards kartica sa odabranim ZedBoard razvojnim sistemom

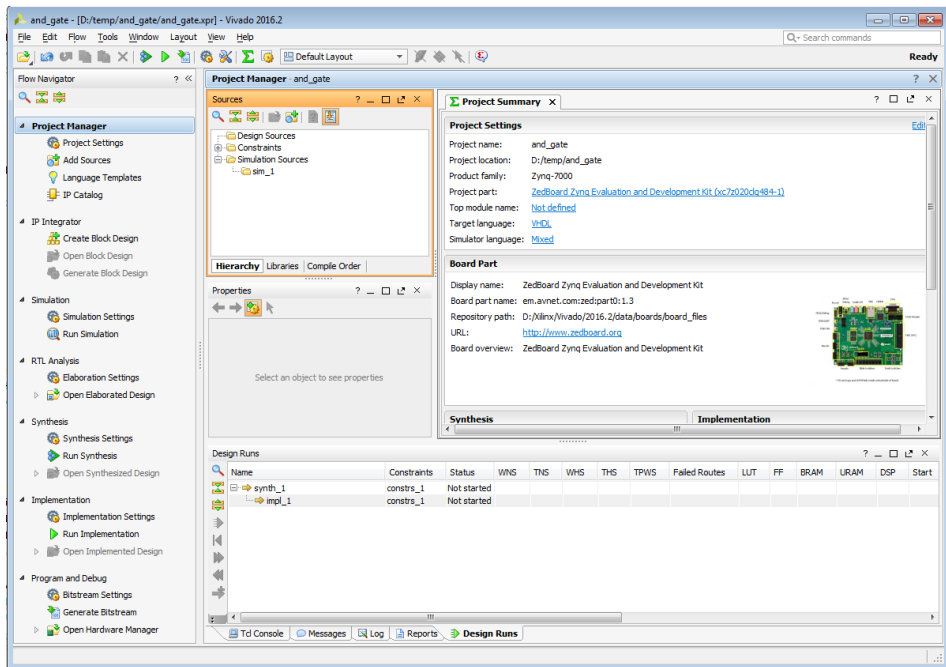
U našem primeru koristićemo ZedBoard Zynq razvojni sistem kompanije Digilent, kao što je prikazano na slici 1.9.

Pritiskom na dugme *Next* završava se proces kreiranja novog projekta. Vivado alat prikazuje glavne karakteristike kreiranog projekta u okviru „New Project Summary“ prozora, prikazanog na slici 1.10.



Slika 1.10. „New Project Summary“ prozor

Ovo je prilika da projektant još jednom proveri da li su sva podešavanja za novi projekat postavljena na željeni način. Ukoliko jesu, pritiskom na dugme *Finish* kreira se novi projekat, a Vivado Design Suite alat se vraća u glavni prozor prikazan na slici 1.11.



Slika 1.11. Glavni prozor Vivado Design Suite alata

Glavni prozor Vivado Design Suite alata sadrži većinu komandi i opcija za izvršavanje svih koraka koji su neophodni prilikom implementacije digitalnog sistema pomoću FPGA komponenti, kao što je ranije bilo objašnjeno i prikazano na slici 1.1.

U okviru „Flow Navigator“ prozora nalazi se većina procesa koje je tipično neophodno izvršiti prilikom implementacije sistema.

U okviru „Project Manager“ grupe nalaze se komande pomoću kojih je moguće:

- Podesiti podešavanja projekta, „Project Settings“ komadna
- Dodati nove izvorne fajlove, „Add Sources“ komanda
- Preuzeti šablone za modelovanje standardnih kombinacionih i sekvencijalnih mreža pomoću VHDL, Verilog i SystemVerilog jezika, „Language Templates“ komanda
- Pretraživati katalog raspoloživih IP jezgara koja se zatim mogu konfigurisati i uključiti u projekat, „IP Catalog“ komanda

U okviru „*IP Integrator*“ grupe nalaze se komande pomoću kojih je moguće:

- Kreirati novi blok dijagram sistema koji se razvija, „*Create Block Design*“ komanda
- Otvoriti postojeći blok dijagram, „*Open Block Design*“ komanda
- Generisati potrebne izvorne fajlove na osnovu kreiranog blok dijagrama, „*Generate Block Design*“ komanda

U okviru *Simulation* grupe nalaze se komande pomoću kojih je moguće:

- Podesiti HDL simulator, „*Simulation Settings*“ komanda
- Pokrenuti odabranu simulaciju, „*Run Simulation*“ komanda

U okviru „*RTL Analysis*“ grupe nalaze se komande pomoću kojih je moguće:

- Podesiti parametre koji se koriste prilikom elaboracije dizajna, „*Elaboration Settings*“ komanda
- Otvoriti i pregledati blok dijagram dizajna koji je nastao nakon elaboracije svih HDL fajlova koji čine projekat, „*Open Elaborated Design*“

U okviru *Synthesis* grupe nalaze se komande pomoću kojih je moguće:

- Podesiti opcije alata za sintezu, „*Synthesis Settings*“ komanda
- Izvršiti proces sinteze hardvera na osnovu HDL izvornih fajlova koji čine projekat, „*Run Synthesis*“ komanda
- Otvoriti i pregledati blok dijagram sintetizovanog dizajna, „*Open Synthesized Design*“ komanda

U okviru *Implementation* grupe nalaze se komande pomoću kojih je moguće:

- Podesiti opcije alata za implementaciju, „*Implementation Settings*“ komanda
- Izvršiti proces implementacije (koraci Translacija, Mapiranje, Rasporadivanje i povezivanje sa slike 1.1) sintetizovanog dizajna, „*Run Implementation*“ komanda
- Otvoriti i pregledati blok dijagram implementiranog dizajna, „*Open Implemented Design*“ komanda

U okviru „*Program and Debug*“ grupe nalaze se komande pomoću kojih je moguće:

- Podesiti podešavanja koja se koriste prilikom generisanja konfiguracionog fajla, „*Bitstream Settings*“ komanda

- Izvršiti proces generisanja konfiguracionog fajla na osnovu implementiranog dizajna, „*Generate Bitstream*“ komanda
- Ostvariti vezu sa FPGA komponentom radi njenog konfigurisanja i eventualno hardverskog testiranja, „*Open Hardware Manager*“ komanda

Centralni deo glavnog prozora Vivado Design Suite alata zauzimaju prozori čiji sadržaj zavisi od režima u kome se trenutno nalazi Vivado alat. Neki od mogućih režima su:

- režim upravljanja projektom,
- režim simulacije,
- režim analize sintetizovanog ili implementiranog dizajna,
- režim programiranja i hardverskog testiranja.

Nakon što se projekat kreira, podrazumevani režim rada je „*Project Manager*“, u kojem se u okviru ovog prozora prikazuje:

- Lista svih izvornih fajlova tekućeg projekta i njihova hijerarhija, *Sources* prozor, kartica *Hierarchy*
- Osobine i svojstva odabranog izvornog fajla, *Properties* prozor
- Podešavana projekta, glavni rezultati sinteze, implementacije i generisanja konfiguracionog fajla, „*Project Summary*“ prozor

Na samom dnu glavnog prozora Vivado Design Suite alata, nalazi se veći broj kartica koje pružaju sledeće informacije:

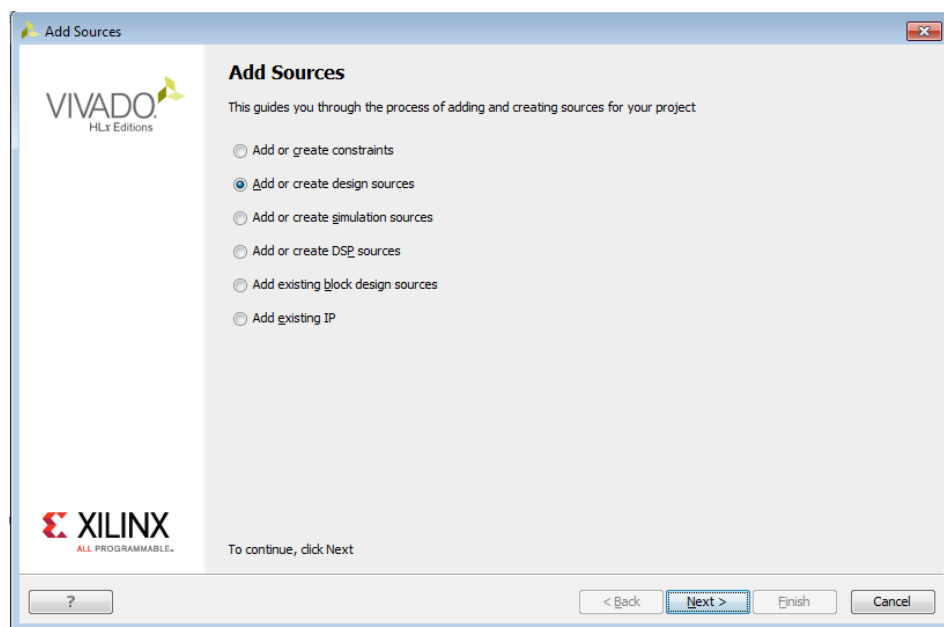
- „*Tcl Console*“ kartica, u kojoj se automatski ispisuju Tcl komande koje se trenutno izvršavaju od strane Vivado alata. Pored toga, u okviru ove kartice moguće je i zadavati Tcl komande koje želimo da Vivado alat izvrši.
- *Messages* kartica, gde se nalaze sve poruke, upozorenja i greške koje je generisao Vivado alat prilikom izvršavanja zadate komande
- *Log* kartica, u okviru koje se nalaze evidencijski fajlovi koji prikazuju sve događaje koji su se desili prilikom procesa sinteze, implementacije i simulacije dizajna
- *Reports* kartica, u okviru koje se nalaze različiti izveštaji generisani od strane različitih Vivado alata u procesu sinteze, implementacije i generisanja konfiguracionog fajla
- „*Design Runs*“ kartica, koja prikazuje trenutno aktivne procese u okviru Vivado alata, kao i njihov trenutni i konačni status

1.4. Dodavanje novog VHDL modela

Nakon što smo kreirali novi projekat, potrebno je da u njega dodamo izvorne fajlove u kojima će se nalaziti sve potrebne informacije o digitalnom sistemu koji je potrebno implementirati. Ovi izvorni fajlovi mogu da sadrže:

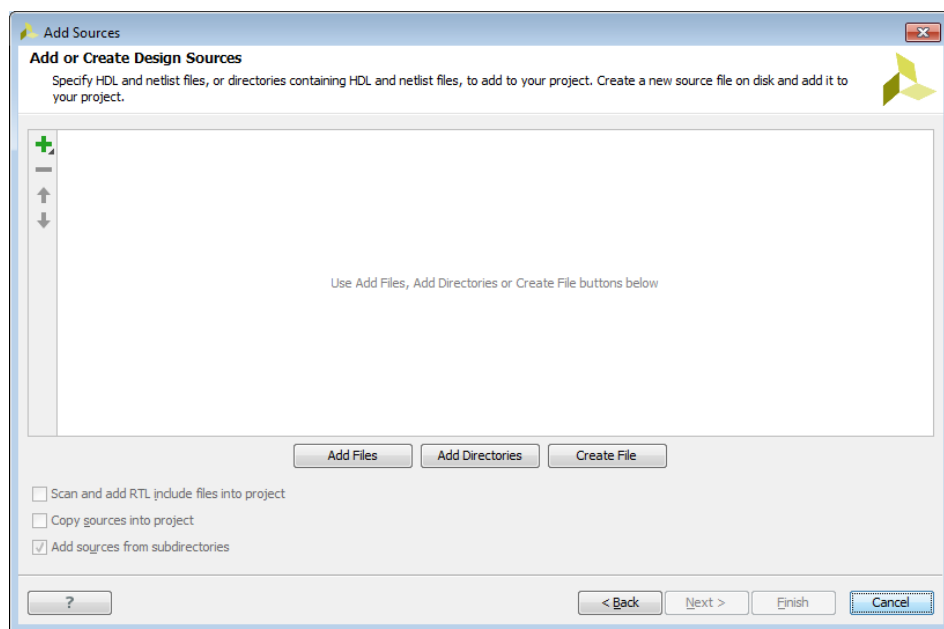
- HDL modele digitalnog sistema koji se projektuje,
- konfiguracije IP modula (prethodno razvijenih digitalnih sistema koje želimo da ponovo koristimo u tekućem projektu) koji će biti korišćeni prilikom implementacije,
- različita prostorna i vremenska ograničenja koja se moraju ispoštovati prilikom implementacije digitalnog sistema od strane Vivado alata

Postupak dodavanja izvornih fajlova započinje pritiskom na „Add Sources“ komandu, unutar „Project Manager“ grupe (koji se nalazi u gornjem desnom uglu „Flow Navigator“ prozora) ili, alternativno, izborom opcije „File/Add Sources...“. Nakon ove akcije pojavljuje se prozor prikazan na slici 1.12.



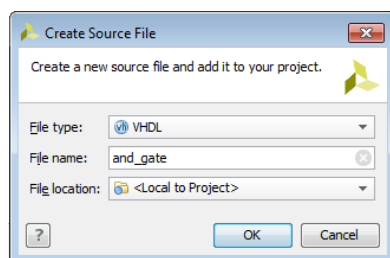
Slika 1.12. „Add Sources“ prozor

Kao što se sa slike 1.12 može videti, možemo uključiti različite vrste izvornih fajlova. U našem primeru kreiraćemo novi izvorni VHDL fajl, u kojem će biti smešten model dvoulaznog I kola, tako da je potrebno izabrati opciju „Add or create design sources“, kao što je prikazano na slici 1.12. Pritiskom na dugme „Next“ otvara se novi prozor, prikazan na slici 1.13.



Slika 1.13. „Add or Create Design Sources“ prozor

Prozor sa slike 1.13 gotovo je identičan sa prozorom sa slike 1.5. Pritiskom na dugme „Create File“ započinje postupak kreiranja novog izvornog HDL fajla, prikazivanjem prozora sa slike 1.14.

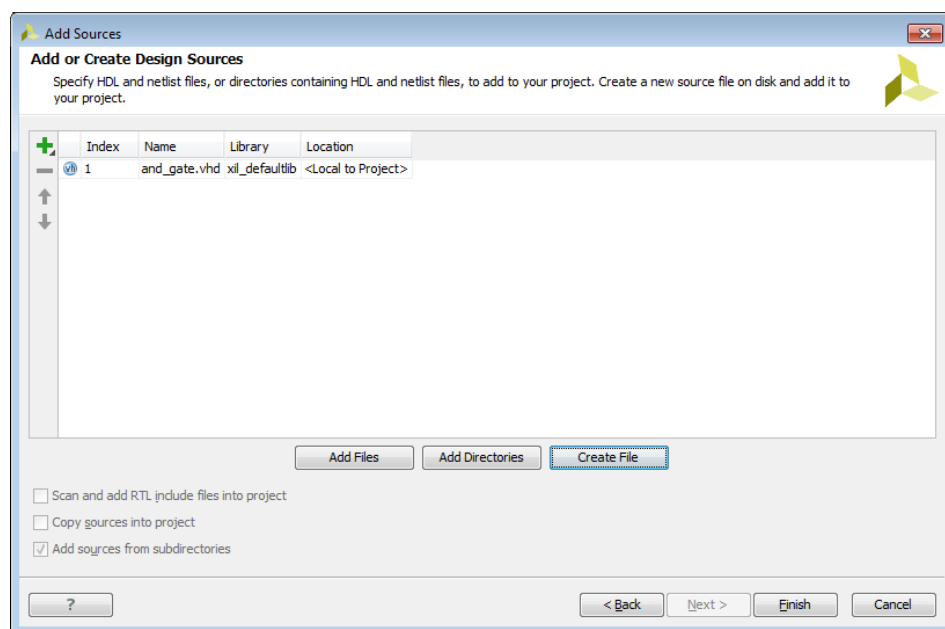


Slika 1.14. „Create Source File“ prozor

U okviru „Create Source File“ prozora specificiraju se glavne karakteristike novog izvornog HDL fajla:

- Polje „File type“ – HDL jezik koji će biti korišćen. Moguće opcije su VHDL, Verilog, SystemVerilog.
- Polje „File name“ – ime HDL fajla koje će biti korišćeno prilikom smeštanja fajla na tvrdi disk
- Polje „File location“ – mesto na disku gde je potrebno smestiti kreirani fajl. Moguća opcije su: „Local to Project“ (fajl će biti smešten u odgovarajući direktorijum u okviru projektnog direktorijuma) ili „Choose Location...“ (korisnik može sam da specificira željenu lokaciju na koju je potrebno smestiti kreirani izvorni fajl).

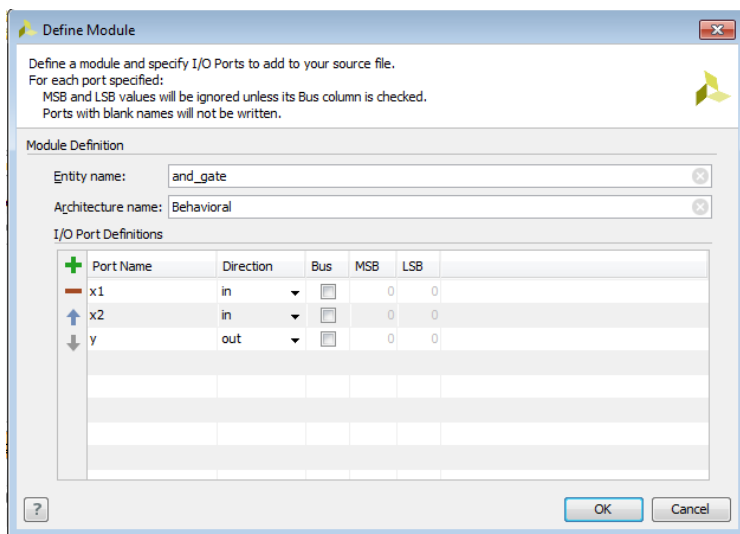
U našem slučaju podešavanja treba da budu jednaka onim koja su prikazana na slici 1.14. Pritiskom na dugme OK, vraćamo se u „Add or Create Design Sources“ prozor, u kome se sada nalazi novokreirani VHDL izvorni fajl, kao što je prikazano na slici 1.15.



Slika 1.15. „Add or Create Design Sources“ prozor, nakon dodavanja izvornog VHDL fajla

Pošto je u našem primeru ovo jedini HDL izvorni fajl unutar projekta, pritiskom na *Finish* dugme završavamo proces dodavanja izvornih fajlova u projekat. Sledeći korak

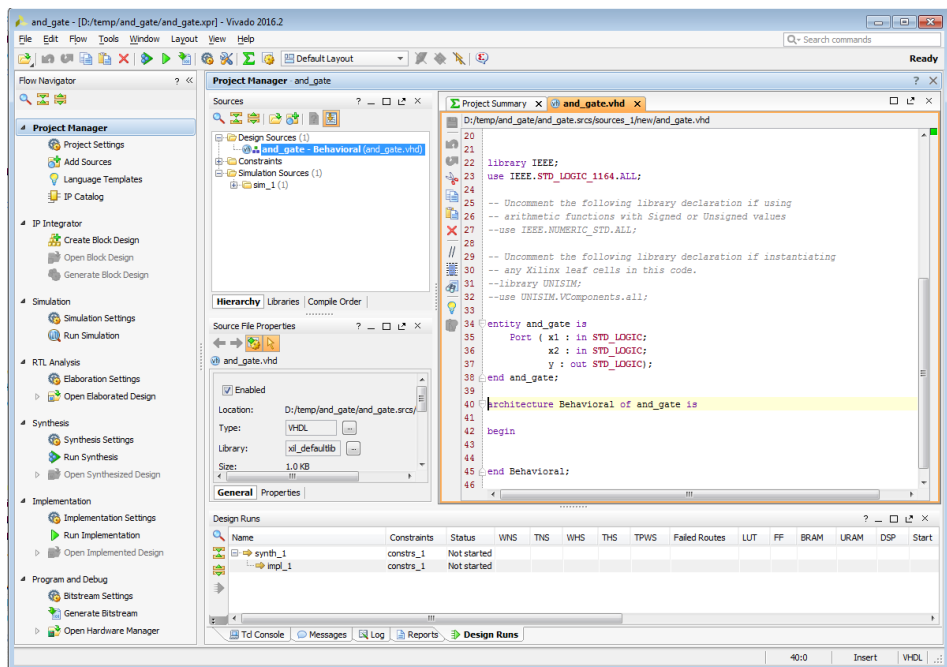
predstavlja specificiranje interfejsa (port mape) modula koji će biti modelovan pomoću dodatog HDL izvornog fajla. U tu svrhu otvara se „Define Module“ prozor prikazan na slici 1.16.



Slika 1.16. „Define Module“ prozor sa potrebnim podešavanjima

U okviru „Define Module“ prozora moguće je definisati interfejs modula koji dodajemo u projekat, kao i ime odgovarajućeg VHDL entiteta („Entity name“) i arhitekturnog tela („Architecture name“) koji će biti automatski kreirani. U našem slučaju potrebna podešavanja prikazana su na slici 1.16. Ime VHDL entiteta biće *and_gate*, isto kao i ime VHDL izvornog fajla koji kreiramo. Ovo je inače standardna praksa. Ime arhitekturnog tela može biti proizvoljno, ali se u praksi najčešće koriste imena poput *behavioral*, *beh*, *rtl* i *synth*, u zavisnosti od vrste VHDL modela koji se razvija.

Interfejs dvoulaznog I kola sastoji se iz dva 1-bitna ulazna porta i jednog 1-bitnog izlaznog porta. Imena portova mogu se birati proizvoljno i u našem slučaju ona su *x1* i *x2* za dva ulazna porta, i *y* za izlazni port. Ime porta zadaje se u polju „Port Name“, a njegov smer u polju *Direction*. Ukoliko je port višebitni (predstavlja magistralu) potrebno je selektovati polje *Bus* i definisati njenu širinu (specificirajući indeks bita najveće i najmanje težine, *MSB* i *LSB*). Nakon što smo definisali sve portove modula čiji model razvijamo, pritiskom na dugme *OK* vraćamo se u glavni prozor Vivado Design Suite alata, čiji sadržaj nakon dodavanja *and_gate.vhd* izvornog fajla je prikazan na slici 1.17.



Slika 1.17. Glavi prozor Vivado Design Suite alata nakon dodavanja novog VHDL izvornog fajla *and_gate.vhd* u projekat

Inspekcijom sadržaja *Hierarchy* kartice u okviru *Sources* prozora možemo videti da je u projekat dodat novi VHDL izvorni fajl, sa imenom *and_gate.vhd*. Dvoklikom na ovaj izvorni fajl u desnom prozoru možemo videti njegov trenutni sadržaj. Ovaj sadržaj kreiran je automatski od strane Vivado Design Suite alata, na osnovu zadatih podešavanja prilikom kreiranja izvornog VHDL fajla. Možemo videti da je kompletna *entity* deklaracija kreirana automatski, i da ona sadrži sve portove koje smo definisali prilikom kreiranja izvornog VHDL fajla (u našem slučaju, to su dva ulazna porta *x1* i *x2*, kao i jedan izlazni port, *y*). Ime entiteta je *and_gate*, upravo ono koje smo specificirali prilikom kreiranja izvornog VHDL fajla. Isto važi i za arhitekturno telo. Sa slike 1.17 takođe možemo primetiti da je arhitekturno telo prazno, što je se moglo i očekivati, jer Vivado alat ne može znati koju funkcionalnost želimo modelovati unutar kreiranog VHDL modula. Ovo je posao dizajnera, i za sada se ne može automatizovati.

Ono što takođe možemo primetiti na slici 1.17, je da se promenio i sadržaj „*Source File Properties*“ prozora (koji se nalazi odmah ispod *Sources* prozora). U ovom prozoru se nalaze se dve kartice:

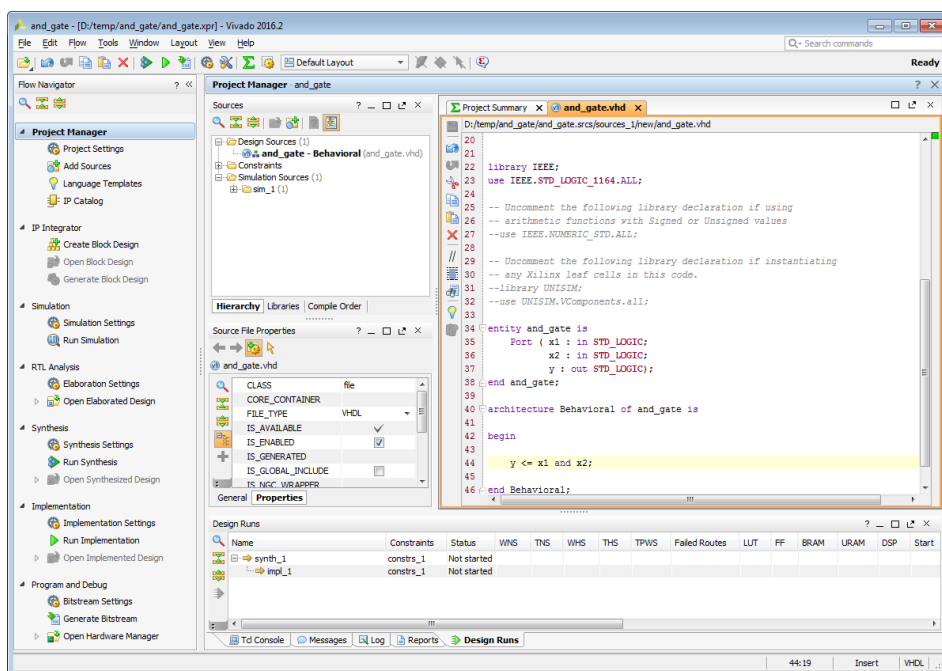
- *General*, koja prikazuje opšte informacije o fajlu koji je trenutno selektovan u *Sources* prozoru i
- *Properties*, koja prikazuje sve osobine selektovanog fajla koje se mogu menjati u okviru Vivado alata

Kao što smo već primetili, trenutni VHDL model unutar *and_gate.vhd* fajla ne modeluje nikakvu funkcionalnost, s'obzirom da je njegovo arhitekturno telo prazno. Funkcionalnost koju želimo da modelujemo jeste funkcionalnost dvoulaznog I logičkog kola. Ova funkcionalnost se vrlo lako može modelovati u VHDL jeziku pomoću samo jedne naredbe konkurentne dodele vrednosti signalu i ugrađenog logičkog operatora *and*. Unutar arhitekturnog tela potrebno je upisati sledeću VHDL naredbu, između *begin* i *end* ključnih reči, unutar *and_gate.vhd* fajla,

$$y \leq x1 \text{ and } x2;$$

Nakon ove izmene, sadržaj *and_gate.vhd* fajla treba da odgovara onom sa slike 1.18.

NAPOMENA: Da bi Vivado Design Suite alat registrovao načinjenu izmenu unutar *and_gate.vhd* fajla, neophodno ga je snimiti. Ovo se postiže izborom opcije *File/Save* ili pritiskom na *Ctrl+S* kombinaciju tastera.



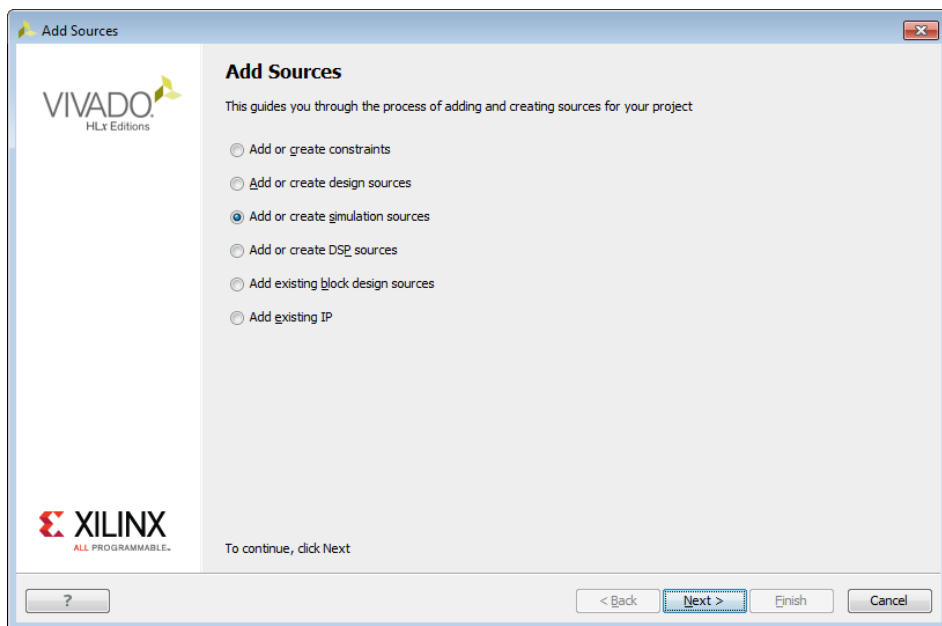
Slika 1.18. Sadržaj VHDL izvornog fajla *and_gate.vhd* nakon dodavanja linije koja modeluje ponašanje dvoulaznog I logičkog kola

1.5. Funkcionalna verifikacija

Pre započinjanja procesa implementacije, potrebno je izvršiti verifikaciju funkcionalne korektnosti razvijenog HDL modela digitalnog elektronskog sistema. U ovoj fazi razvoja jedino je moguće izvršiti funkcionalnu (bihevioralnu) verifikaciju, jer još uvek nam nisu poznata stvarna kašnjenja unutrašnjih signala sistema koji projektujemo. Kao što je ranije već rečeno, stvarna kašnjenja unutrašnjih signala biće poznata tek nakon što se završi faza raspoređivanja i povezivanja, koja je poslednji korak prilikom implementacije dizajna pomoću odabrane tehnologije (FPGA ili ASIC). Vivado Design Suite alat nam omogućava da izvršimo funkcionalnu verifikaciju pomoću simulacije, pokretanjem komande „Run Simulation“ iz „Simulation“ grupe komandi.

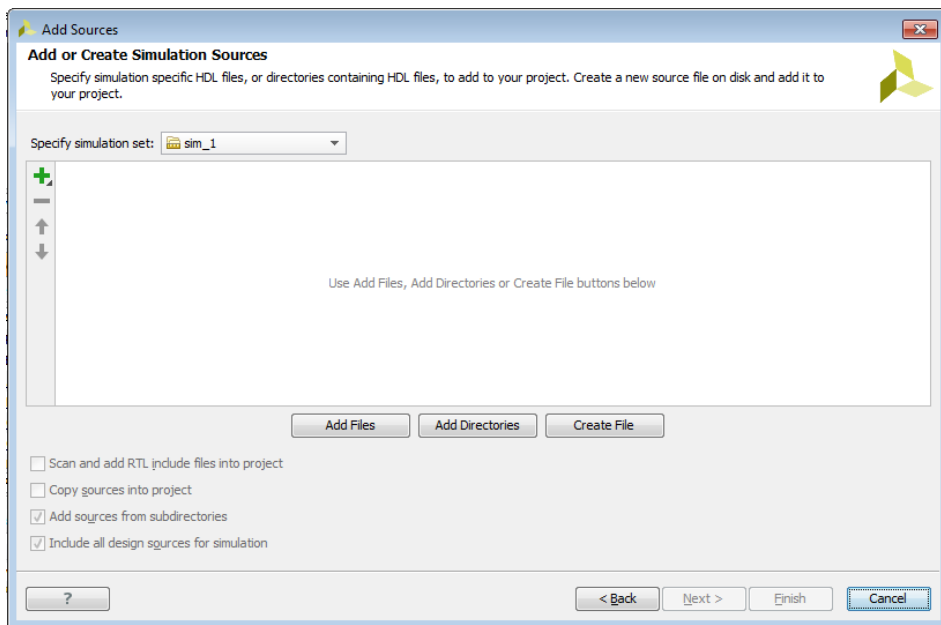
Ipak, pre nego što pokrenemo funkcionalnu simulaciju VHDL modela dvoulaznog I logičkog kola, neophodno je dodati još jedan VHDL fajl, takozvani testbenč (*testbench*) fajl. Testbenč je HDL model koji nije namenjen za implementaciju, već isključivo za verifikaciju digitalnog sistema koji se projektuje. Unutar testbenča nalazi se logika za generisanje stimulusa (pažljivo odabranih sekvenci vrednosti za sve ulazne portove digitalnog sistema koji se verifikuje) i, opciono, logika za proveru ispravnosti generisanih vrednosti na izlaznim portovima digitalnog elektronskog sistema koji se verifikuje.

Dodavanje VHDL testbenč fajla, unutar Vivado Design Suite alata, izvodi se na isti način kao i dodavanje izvornog VHDL fajla koji sadrži model digitalnog sistema koji se projektuje. Izborom komande „Add Sources“ ponovo se otvara „Add or Create Design Sources“ prozor. Međutim, za razliku od dodavanja izvornog VHDL model fajla, ovoga puta je potrebno izabrati opciju „Add or create simulation sources“, kao što je prikazano na slici 1.19.



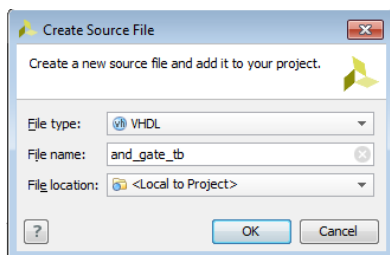
Slika 1.19. „Add Sources“ prozor sa potrebnim podešavanjima u slučaju kreiranja novog VHDL testbenč fajla

Pritiskom na *Next* dugme otvara se „Add or Create Simulation Sources“ prozor, prikazan na slici 1.20. Izgled „Add or Create Simulation Sources“ prozora identičan je sa izgledom „Add or Create Design Sources“ prozora sa slike 1.13, te nećemo detaljnije objašnjavati značenje pojedinih komandi. Jedinu novinu predstavlja „Specify simulation set“ polje, u okviru kojega je moguće specificirati ime simulacionog seta u koji će biti uključeni novi testbenč fajlovi. U našem primeru ovo polje treba ostaviti nepromenjeno.



Slika 1.20. „Add or Create Simulation Sources“ prozor

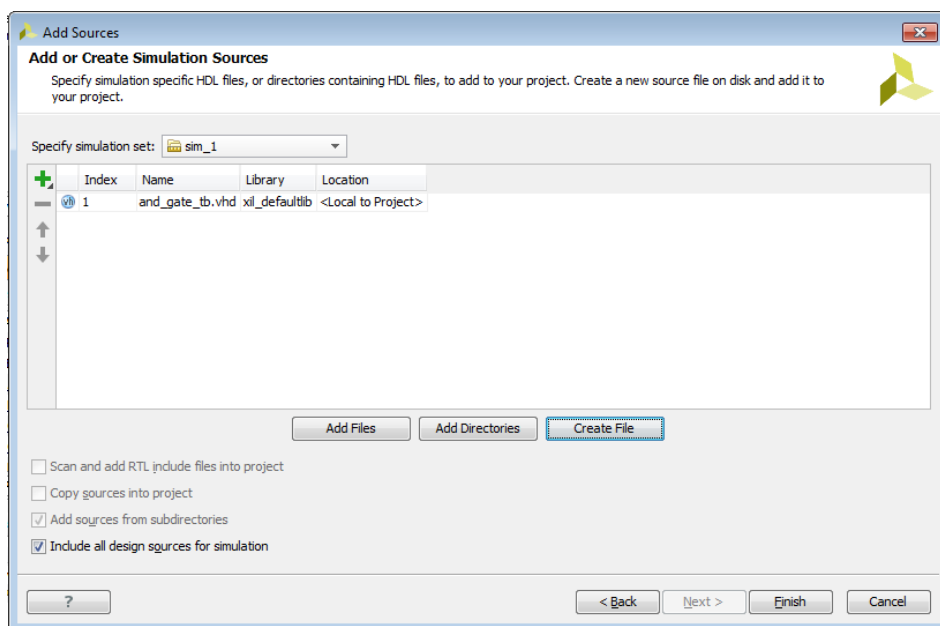
Pritiskom na „Create File“ dugme započinje se proces kreiranja i dodavanja novog testbenč fajla, pojavljivanjem prozora sa slike 1.21.



Slika 1.21. „Create Source File“ prozor

Kao i u slučaju dodavanja VHDL model fajla, potrebno je specificirati ime za VHDL testbenč fajl koji će biti kreiran. Iako ovo ime može biti proizvoljno, praksa je da ono sadrži ime VHDL entiteta kome je testbenč fajl koji se kreira asociran, praćeno sufiksom `_tb`, koji označava da je reč o testbenč fajlu a ne o dizajn fajlu. U našem slučaju, pošto kreiramo testbenč fajl za VHDL model dvoulaznog I logičkog kola čiji entitet je ranije nazvan `and_gate`, prateći prethodno pomenutu konvenciju, ime VHDL testbenč fajla će biti `and_gate_tb.vhd`. Nakon što smo podesili sva podešavanja,

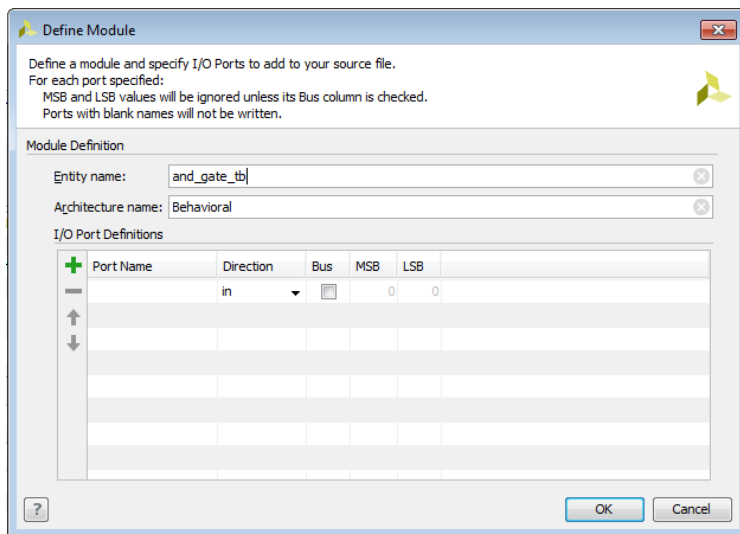
pritiskom na dugme *OK*, vraćamo se u „*Add or Create Design Sources*“ prozor, čiji sadržaj sada pokazuje da se u okviru *sim_1* simulacionog seta sada nalazi *and_gate_tb.vhd* testbenč fajl, kao što se može videti sa slike 1.22.



Slika 1.22. „Add or Create Simulation Sources“ prozor, nakon dodavanja and_gate_tb.vhd testbenč fajla

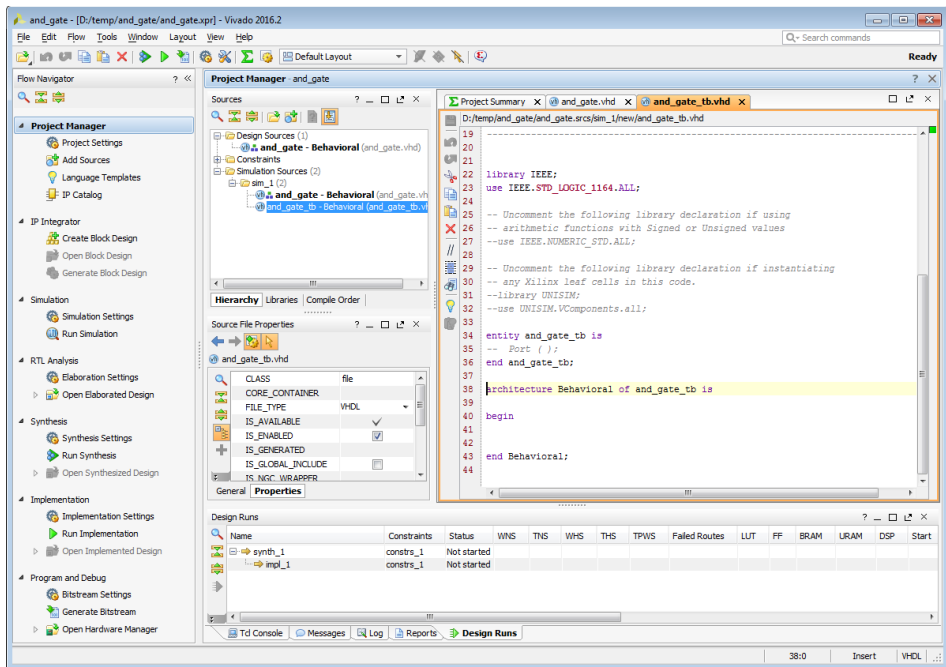
Pritiskom na *Finish* dugme prelazimo na poslednji korak kreiranja VHDL testbenč fajla, koji se odnosi na specificiranje njegovog interfesja, pomoću prozora prikazanog na slici 1.23.

Kako je reč o testbenč fajlu, on ne bi trebao da ima niti jedan ulazni niti izlazni port, te je njegovu port mapu potrebno ostaviti praznom, kao što je prikazano na slici 1.23.



Slika 1.23. „Define Module“ prozor sa potrebnim podešavanjima u slučaju kreiranja novog VHDL testbenč fajla

Pritiskom na OK dugme Vivado Design Suite alat će kreirati VHDL testbenč fajl i vratiti u osnovni prozor, čiji sadržaj je prikazan na slici 1.24. Ono što se može videti sa slike 1.24 je da je u okviru Sources prozora, unutar *sim_1* foldera dodat jedan novi izvorni fajl, VHDL testbenč fajl koji smo upravo kreirali, *and_gate_tb.vhd*. Sa slike takođe možemo primetiti da se pored *and_gate_tb.vhd* VHDL testbenč fajla, u okviru *sim_1* foldera nalazi i *and_gate.vhd* VHDL dizajn fajl, koji se nalazi na istom nivou hijerarhije sa *and_gate_tb.vhd* VHDL testbenč fajlom. *Hierarchy* kartica ne samo da prikazuje koji sve izvorni čine tekući projekat, već takođe prikazuje i njihovu hijerarhijsku organizaciju (otuda i njeno ime, *Hierarchy* kartica). VHDL dizajn fajl *and_gate.vhd* trebalo bi da se nalazi na nižem hijerarhijskom nivou od *and_gate_tb.vhd* testbenč fajla. Ovo još uvek nije slučaj jer je *and_gate_tb.vhd* testbenč fajl prazan, što se može videti ako se prikaže trenutni sadržaj *and_gate_tb.vhd* fajla dvoklikom na njega, pa Vivado Design Suite alat nema načina kako da sazna ovu informaciju.



Slika 1.24. Izgled galvnog prozora Vivado Design Suite alata nakon što je kreiran novi VHDL testbenč fajl

Prazno arhitekturno telo `and_gate_tb.vhd` VHDL testbenč fajla potrebno je zameniti sa sledećim arhitekturnim telom

```

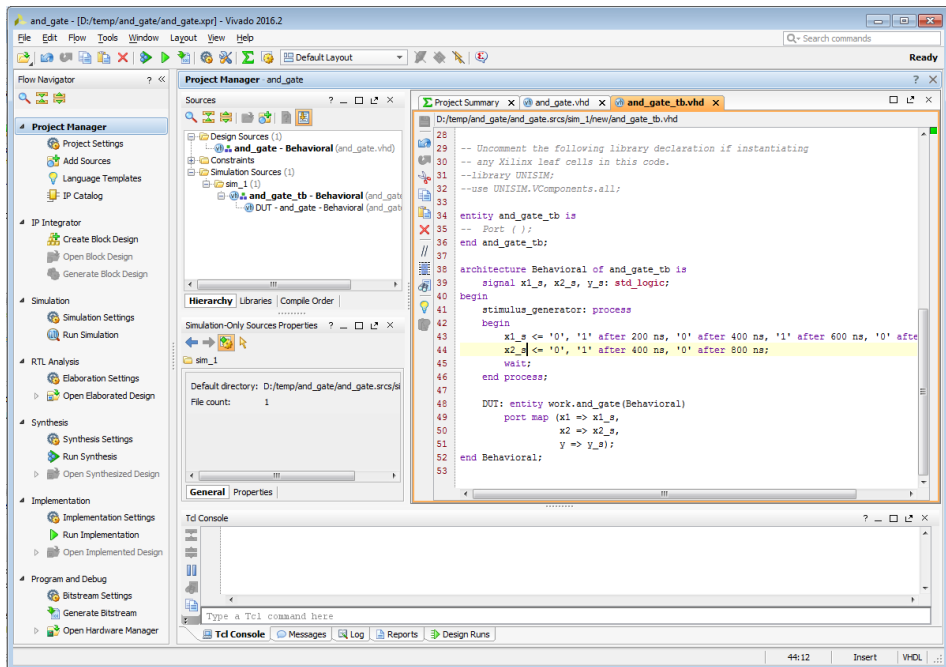
architecture Behavioral of and_gate_tb is
  signal x1_s, x2_s, y_s: std_logic;
begin
  stimulus_generator: process
  begin
    x1 <= '0', '1' after 200 ns, '0' after 400 ns, '1' after 600 ns, '0' after 800 ns;
    x2 <= '0', '1' after 400 ns, '0' after 800 ns;
    wait;
  end process;

  DUT: entity work.and_gate(Behavioral)
    port map (x1 => x1_s,
              x2 => x2_s,
              y => y_s);
end Behavioral;

```

NAPOMENA: Da bi Vivado Design Suite alat registrovao načinjenu izmenu unutar `and_gate_tb.vhd` VHDL testbenč fajla, neophodno ga je snimiti. Ovo se postiže izborom opcije *File/Save* ili pritiskom na Ctrl+S kombinaciju tastera.

Nakon učinjenih modifikacija, sadržaj glavnog prozora Vivado Design Suite alata se menja, kao što je prikazano na slici 1.25.



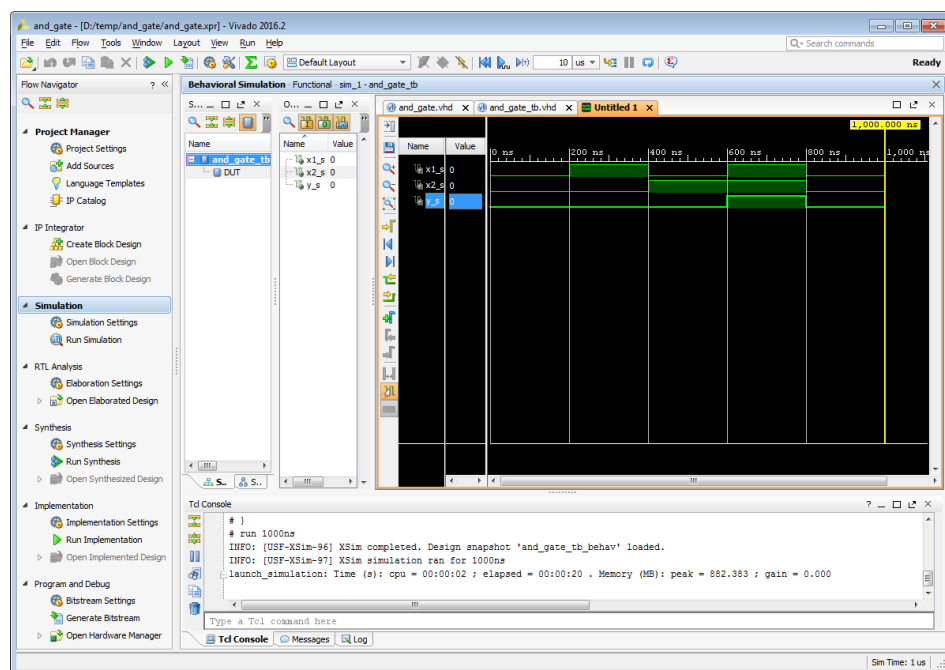
Slika 1.25. Izgled glavnog prozora Vivado Design Suite alata nakon što je modifikacije VHDL testbenč fajla

Posebno obratite pažnju na novu organizaciju fajlova unutar *sim_1* foldera. Sada se može videti očekivna hijerarhijska struktura *and_gate_tb.vhd* i *and_gate.vhd* fajlova. *and_gate.vhd* fajl sada se nalazi na hijerarhijski nižem nivou od *and_gate_tb.vhd* fajla, kao što je i očekivano.

Konačno možemo pristupiti funkcionalnoj simulaciji *and_gate* VHDL modela. Ovo se postiže pristikom na „Run Simulation“ komandu, unutar *Simulation* grupe i izborom opcije „Run Behavioral Simulation“. Pritiskom na navedeno dugme, pokreće se ugrađeni Vivado simulator koji simulira ponašanje *and_gate* VHDL modela tokom specificiranog intervala vremena.

NAPOMENA: Podrazumevana vrednost simulacije iznosi 1000 ns, i može se promeniti izborom komande „Simulation Settings“ i izmenom vrednosti *xsim.simulate.runtime* polja koje se nalazi unutar *Simulation* kartice.

Nakon što se završi proces simulacije, automatski će se otvoriti novi prozor Vivado simulatora, koji služi za analizu rezultata simulacije. U našem slučaju, Vivado simulator prozor imaće izgled prikazan na slici 1.26.



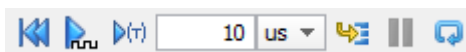
Slika 1.26. Izgled Vivado simulatora prozora nakon završene simulacije *and_gate* VHDL modela

Vivado simulator prozor organizovan je u nekoliko celina. Centralno mesto zauzima prozor u kojem su prikazani talasni oblici odabranih signala, koji su generisani kao rezultat izvršavanja simulacije sistema u navedenom vremenskom periodu. Na slici 1.26 u okviru ovog prozora možemo videti talasne oblike za tri signala: *x1_s*, *x2_s* i *y_s*. Upoređivanjem talasnih oblika za signale *x1_s* i *x2_s*, možemo zaključiti da oni odgovaraju transakcijama na ova dva signala koje smo specificirali u okviru *stimulus_generator* procesa, u okviru *and_gate_tb.vhd* VHDL testbenč fajla (videti sliku 1.25). Talasni oblik signala *y_s* nije bio eksplicitno definisan, jer on zapravo predstavlja odziv VHDL modela dvoulaznog I logičkog kola na generisani stimulus (talasne oblike signala *x1_s* i *x2_s*), koji će biti izračunat u procesu funkcionalne simulacije od strane simulatora. Analizom talasnog oblika signala *y_s*, verifikacioni inženjer treba da utvrdi da li je zahtevana funkcionalnost dvoulaznog I logičkog kola korektno modelovana unutar *and_gate.vhd* VHDL modela. Sa slike 1.26 može se videti da je u ovom slučaju zahtevana funkcionalnost korektno modelovana, jer signal *y_s*, izlaz dvoulaznog I logičkog kola, ima vrednost logičke nule dokle god barem jedan od ulaznih signala, *x1_s* ili *x2_s*, ima vrednost logičke nule. Tek kada su oba ulazna signala

$x1_s$ i $x2_s$, na logičkoj jedinici, izlazni signal y_s takođe postaje jednak logičkoj jedinici. Ovo je očekivano ponašanje dvoulaznog I logičkog kola.

Pored prozora sa talasnim oblicima, levo od njega nalaze se dva dodatna prozora. *Scopes* prozor (krajnji prozor levo u okviru „*Behavioral Simulation*“ prozora) prikazuje hijerarhijsku strukturu sistema koji se simulira, dok *Objects* prozor (srednji prozor) prikazuje listu objekata koji su vidljivi unutar trenutno selektovane instance ili procesa unutar *Scopes* prozora. Ove objekte možemo prebaciti u prozor sa talasnim oblicima, kako bismo mogli da vidimo i njihove talasne oblike koji su izračunati u procesu simulacije.

Iznad ovih prozora nalazi se lista kontrola pomoću kojih se može upravljati tokom simulacije. Najvažnije kontrole prikazane su na slici 1.27.



Slika 1.27. Najvažnije kontrole za upravljanje tokom simulacije

Funkcija pojedinih kontrola sa slike 1.27 je sledeća:

-  *Restart* dugme – pritiskom na ovo dugme čitav proces simulacije započinje iz početka, inicijalizacijom sistema, i simuliranjem njegovog ponašanja tokom specificiranog intervala vremena, pomoću „*Simulation time*“ polja.
-  „*Run All*“ dugme – pritiskom na ovo dugme pokreće se simulacija koja traje sve dok ima događaja u sistemu koji se simulira. Ovakav način pokretanja simulacije nije preporučljiv za većinu modela digitalnih sistema, jer u većini sistema postoje periodični događaji (na primer, promene vrednost klock signala) zbog kojih se simulacija nikada neće završiti.
-  „*Run for the time specified on the toolbar*“ dugme – pritiskom na ovo dugme simulira se ponašanje sistema u toku specificiranog intervala vremena pomoću „*Simulation time*“ polja.
-  „*Simulation time*“ polje – u ovom polju navodi se vremenski period tokom kojega se želi simulirati ponašanje sistema.
-  *Step* dugme – pritiskom na ovo dugme proces simulacije traje do nailaska na prvu liniju unutar nekog od VHDL fajlova koji čine tekuće verifikaciono okruženje na kojoj je postavljena tačka prekida (*breakpoint*).
-  *Break* dugme – pritiskom na ovo dugme prekida se proces simulacije i prikazuju se talasni oblici signala koji su odsimulirani do trenutka prekida simulacije.
-  *Re-launch* dugme – pritiskom na ovo dugme ponovo se inicijalizuje čitav ISim simulator. Inicijalizacija podrazumeva ponovno kompajliranje svih

VHDL fajlova koji čine tekuće verifikaciono okruženje, učitavanje kompajliranih struktura u simulator, i simulaciju ponašanja sistema tokom vremena specificiranog u „*Simulation time*“ polju.

1.6. Sinteza i implementacija

Nakon uspešno završene funkcionalne verifikacije, HDL model digitalnog elektronskog sistema koji želimo da realizujemo trebalo bi da je funkcionalno korektan (ukoliko je korektno izvršena funkcionalna verifikacija), te se može pristupiti njegovoj sintezi i implementaciji. Kao što je u uvodnom delu već naglašeno, proces realizacije HDL modela digitalnog elektronskog sistema pomoću FPGA komponente je vrlo složen, i sastoji se iz sledećih faza:

- sinteze,
- transliranja, mapiranja, raspoređivanja i povezivanja, koje zajedno čine fazu implementacije,
- generisanja konfiguracionog fajla.

Svako od ovih faza asocirana je jedna grupa akcija unutar Vivado „*Flow Navigator*“ prozora:

- „*Synthesize*“ grupa - proces sinteze
- „*Implementation*“ grupa - procesi transliranja, mapiranja, raspoređivanja i povezivanja
- „*Program and Debug*“ grupa - proces generisanja konfiguracionog fajla

Struktura akcija koje su raspoložive unutar svake od ovih grupa je skoro identična. Svaka grupa ima jednu akciju koja se može iskoristiti za konfigurisanje odgovarajućih alata koji će biti korišćeni („*Synthesis Settings*“, „*Implementation Settings*“ i „*Bitstream Settings*“ akcije), zatim je tu akcija pomoću koje se zapravo izvršava odgovarajući proces transformacije dizajna („*Run Synthesis*“, „*Run Implementation*“ i „*Generate Bitstream*“ akcije). U okviru koraka sinteze i implementacije, postoje i akcije koje dozvoljavaju da se provere rezultati izvođenja ovih procesa („*Open Synthesized Design*“ i „*Open Implemented Design*“ akcije), dok u koraku programiranja postoji akcija pomoću koje je moguće ostvariti vezu sa FPGA komponentom i sprovesti sam postupak programiranja i eventualnog testiranja („*Open Hardware Manager*“ akcija).

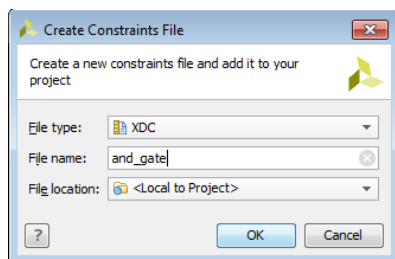
Pre započinjanja procesa sinteze i implementacije, neophodno je u projekat dodati još jedan dizajn fajl. Ovoga puta nije reč o HDL modelu, niti o testbenč fajlu, već je reč o fajlu u kome su specificirana različita ograničenja koja Vivado Design Suite alat mora

ispoštovati prilikom sinteze i implementacije dizajna. Ova ograničenja mogu se grupisati u dve grupe:

- Ograničenja u pogledu željene brzine rada sistema koji se implementira – ovaj tip ograničenja se najčešće svodi na specificiranje minimalne učestanosti za svaki od klock signala koji su prisutni unutar digitalnog elektronskog sistema koji se implementira.
- Prostornih ograničenja – specificiranja nožice FPGA kola na koju treba da bude doveden svaki od signala koji čine interfejs digitalnog elektronskog sistema koji se implementira.

Fajl sa ograničenjima se u okviru Vivado Design Suite alata dodaje kao i bilo koji drugi dizajn fajl. Pritiskom na „Add Sources“ komandu ponovo se pojavljuje „Add Sources“ prozor. Ovog puta potrebno je selektovati „Add or create constraints“ grupu izvornih fajlova, i kliknuti na dugme *Next*, nakon čega će se pojaviti „Add or Create Constraints“ prozor, kao i u slučaju dodavanja novog HDL izvornog fajla u projekat. Jedina razlika je što ovaj put postoji polje „Specify constraint set“ pomoću kojega je moguće definisati ime skupa ograničenja kojem će biti pridružen fajl sa ograničenjima koji kreiramo. U našem slučaju možemo koristiti predloženo ime *constrs_1*. Ostale opcije imaju isto značenje kao i u slučaju „Add or Create Design Sources“ prozora, objašnjenog ranije.

Kako mi želimo da kreiramo novi fajl sa ograničenjima, pa je potrebno pritisnuti dugme „Create File“. Nakon pritiska ovog dugmeta pojavljuje se „Create Constraints File“ prozor, prikazan na slici 1.28.



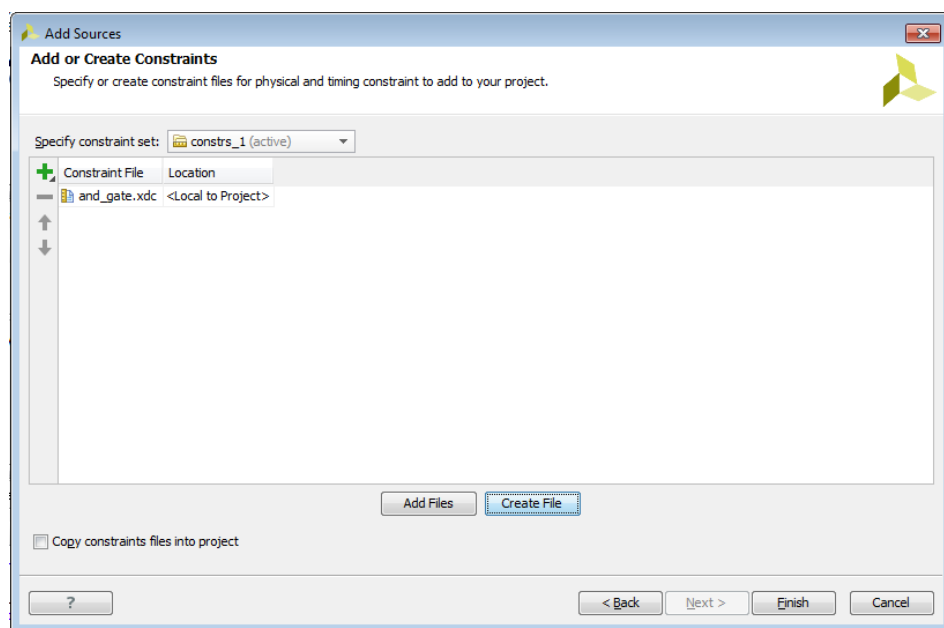
Slika 1.28. „Create Constraints File“ prozor

Sadržaj „Create Constraints File“ prozora sličan je sadržaju „Create Source File“ prozora. U okviru „Create Constraints File“ prozora specificiraju se glavne karakteristike novog fajla sa ograničenjima:

- Polje „File type“ – jezik za specifikaciju ograničenja koji će biti korišćen. Moguće opcije su XDC.

- Polje „*File name*“ – ime fajla sa ograničenjima koje će biti korišćeno prilikom smeštanja fajla na tvrdi disk
- Polje „*File location*“ – mesto na disku gde je potrebno smestiti kreirani fajl sa ograničenjima. Moguća opcije su: „*Local to Project*“ (fajl će biti smešten u odgovarajući direktorijum u okviru projektnog direktorijuma) ili „*Choose Location...*“ (korisnik može sam da specificira željenu lokaciju na koju je potrebno smestiti kreirani fajl).

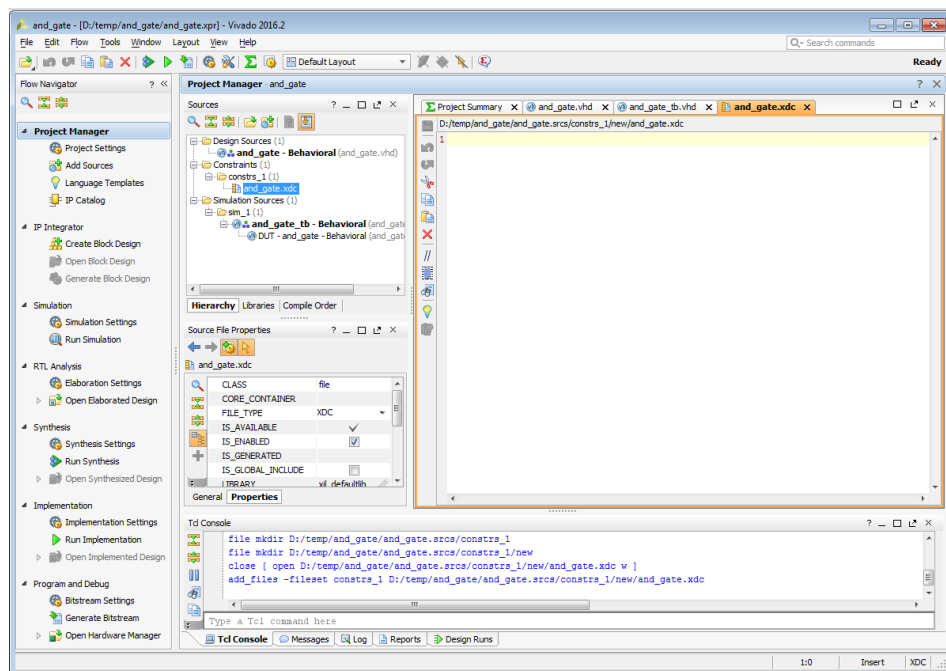
U našem slučaju podešavanja treba da budu jednaka onim koja su prikazana na slici 1.28. Pritiskom na dugme *OK*, vraćamo se u „*Add Constraints*“ prozor, u kome se sada nalazi novokreirani XDC fajl, kao što je prikazano na slici 1.29.



Slika 1.29. „*Add or Create Constraints*“ prozor nakon dodavanja *and_gate.xdc* fajla sa ograničenjima

Kada smo zadovoljni sa podešavanjima, pritiskom na dugme *Finish* vraćamo se u glavni prozor Vivado Design Suite alata. Obratite pažnju da je u okviru *Hierarchy* kartice, unutar *Constraints* foldera dodat novi fajl sa ograničenjima, *and_gate.xdc*. Dvoklikom na ovaj fajl prikazuje se njegov trenutni sadržaj, koji je prazan, što je bilo i očekivano jer Vivado Design Suite alat nema načina da automatski sazna ograničenja koja želimo da definišemo. Ova ograničenja mora uneti projektant, koristeći *Xiling Design*

Constraints sintaksu. Glava 9, „Analiza dizajna i definisanje ograničenja“, posvećena je problematici definisanja ograničenja dizajna.



Slika 1.30. Glavni prozor Vivado alata i početni sadržaj and_gate.xdc fajla sa ograničenjima nakon njegovog kreiranja i uključivanja u projekat

U našem slučaju u fajl sa ograničenjima moramo uneti sledeća prostorna ograničenja:

- Poziciju FPGA nožice na koju će biti doveden ulazni signal x1,
- Poziciju FPGA nožice na koju će biti doveden ulazni signal x2,
- Poziciju FPGA nožice na koju će biti doveden izlazni signal y.

U našem primeru nemamo ni jedan klock signal u sistemu, jer je reč o dvoulaznom I logičkom kolu, te naš fajl sa ograničenjima neće sadržati ni jedno vremensko ograničenje.

Da bi smo ih mogli lako kontrolisati, ulazne signale x1 i x2, povezaćemo sa odgovarajućim prekidačima koji su prisutni na odabranoj razvojnoj pločici. Na ovaj način ćemo vrlo lako moći da menjamo njihove vrednosti (sa logičke nule na logičku jedinicu i obrnuto), što će nam omogućiti da pored izvršene funkcionalne verifikacije sprovedemo i hardversko testiranje sistema nakon što on bude implementiran unutar

FPGA kola. Iz istog razloga, izlazni signal *y* povežemo sa LED diodom koja je takođe prisutna na razvojnom sistemu, kako bismo lako mogli videti koji je logički nivo trenutno prisutan na izlazu (logička nula → LED dioda ne svetli, logička jedinica → LED dioda svetli).

Pozicije FPGA nožica na koje su spolja povezani prekidači i LED dioda zavise od odabranog razvojnog sistema koji će biti korišćen za testiranje. U našem slučaju reč je o ZedBoard razvojnom sistemu kompanije Digilent. Konsultovanjem dokumentacije koja dolazi sa ovim razvojnim sistemom, a može se pronaći i na Internetu (dokument „*ZedBoard Hardware User's Guide*“), mogu se pronaći lokacije FPGA nožica na koje su povezana dva prekidača i jedna LED dioda:

- **Povezivanje prekidača**

Na ZedBoard razvojnom sistemu postoje i prekidači i tasteri („*ZedBoard Hardware User's Guide*“, poglavlje 2.7 „*User I/O*“, odeljci 2.7.1 „*User Push Buttons*“ i 2.7.2 „*User DIP Switches*“) koji bi se mogli iskoristiti za ovu namenu. Mi ćemo koristiti dva prekidača koji nam stoje na raspolaganju, konkretno prekidače SW0 i SW1.

U tom slučaju ulazni port *x1* potrebno je mapirati na FPGA nožicu sa oznakom F22, a ulazni port *x2* potrebno je mapirati na FPGA nožicu sa oznakom UG22 („*ZedBoard Hardware User's Guide*“, poglavlje 2.7 „*User I/O*“, odeljak 2.7.2 „*User DIP Switches*“, tabela 13 na strani 19).

- **Povezivanje LED diode**

Na ZedBoard razvojnom sistemu takođe postoje i LED diode („*ZedBoard Hardware User's Guide*“, poglavlje 2.7 „*User I/O*“, odeljak 2.7.2 „*User LEDs*“) koje se mogu iskoristiti za ovu namenu. Mi ćemo koristiti LED diodu LD0.

U tom slučaju izlazni port *y* potrebno je mapirati na FPGA nožicu sa oznakom T22 („*ZedBoard Hardware User's Guide*“, poglavlje 2.7 „*User I/O*“, odeljak 2.7.2 „*User LEDs*“, tabela 14 na strani 20).

Koristeći ove informacije i poštujući XDC sintaksu za opis ograničenja kompanije Xilinx, u *and_gate.xdc* fajl sa ograničenjima potrebno je uneti sledeće linije

```
set_property PACKAGE_PIN F22 [get_ports x1];
set_property PACKAGE_PIN G22 [get_ports x2];
set_property PACKAGE_PIN T22 [get_ports y];

set_property IOSTANDARD LVCMOS25 [get_ports x1];
set_property IOSTANDARD LVCMOS25 [get_ports x2];
set_property IOSTANDARD LVCMOS33 [get_ports y];
```

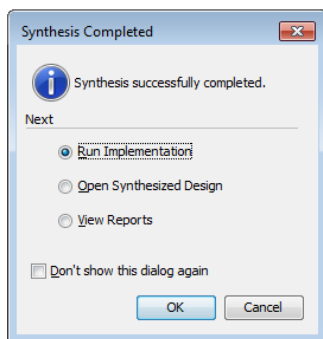
Gornje tri linije povezuju ulazne i izlazne portove našeg digitalnog sistema sa odgovarajućim nožicama FPGA kola, na koja su sa spoljne strane povezani prekidači i

LED dioda. Tako prva linija nalaže Vivado Design Suite alatu da prilikom implementacije ulazni port *x1* fizički poveže sa FPGA nožicom F22. Sa druge strane, vezom na štampanoj ploči ZedBoard razvojnog sistema, jedan prekidač je fizički povezan za nožicom FPGA kola sa oznakom F22. Na ovaj način će svaka promena logičkog nivoa na nožici F22, kao posledica promene položaja prekidača, biti odslikana i u promeni vrednosti ulaznog porta *x1*.

Preostale tri linije u okviru *and_gate.xdc* fajla definišu ulazno/izlazne električne standarde koje želimo da koristimo na svakoj od nožica na koju je mapiran neki od ulaznih, odnosno izlaznih portova dizajna.

NAPOMENA: Nakon što smo u fajl sa ograničenjima uneli gornje tri linije potrebno ga je snimiti, što se može uraditi pritiskom na kombinaciju tastera *Ctrl+S* ili izborom opcije iz menija *File/Save*. Detaljno objašnjenje načina zadavanja različitih ograničenja, kao i sintaksa jezika za specifikaciju ograničenja, biće razmatrani u glavi 9 ovog udžbenika.

Nakon kreiranja fajla sa ograničenjima, koja će biti korišćena prilikom realizacije digitalnog elektronskog sistema pomoću odabrane FPGA komponente, možemo započeti sam proces realizacije. Prvi korak u procesu realizacije je da se VHDL model digitalnog elektronskog sistema sintetizuje, odnosno prevede u netlistu sastavljenu od elementarnih logičkih kapija, flip-flopova, lečeva, itd. Ova netlista poznata je pod imenom *gate-level* netlista. Klikom na komandu „*Run Synthesis*“ u okviru *Synthesis* grupe započinje process sinteze. Nakon što se process sinteze završi, Vivado alat će nas obavestiti o tome izbacivanjem „*Synthesis Completed*“, čiji izgled je prikazan na slici 1.31.

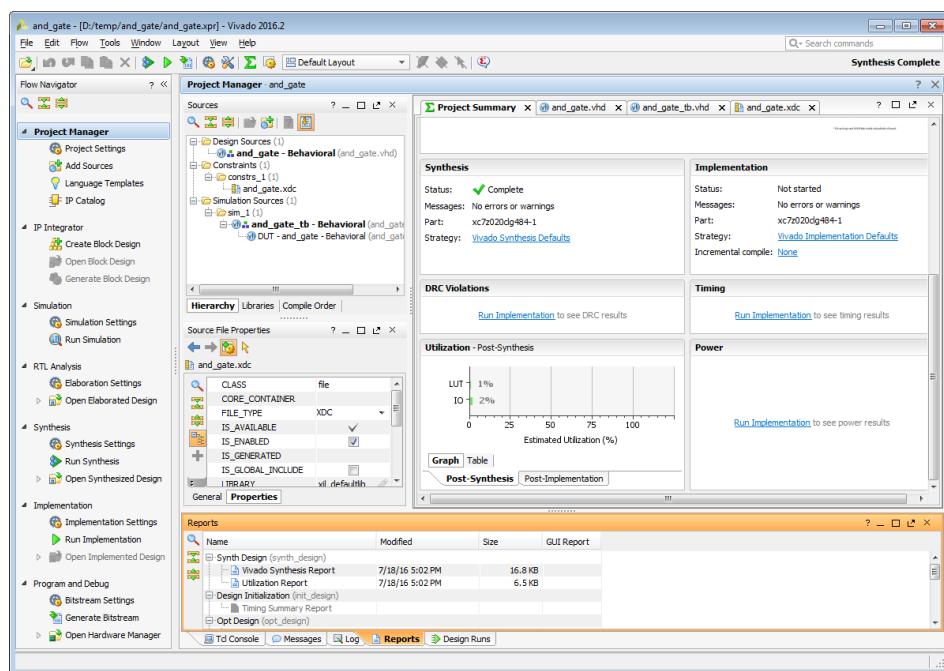


Slika 1.31. „*Synthesis Completed*“ prozor koji se pojavljuje nakon što je završena faza sinteze dizajna

„*Synthesis Completed*“ prozor nudi tri akcije:

- „Run Implementation“ – pomoću koje je moguće pokrenuti proces implementacije
- „Open Synthesized Design“ – pomoću koje je moguće otvoriti sintetizovani dizajn i izvršiti njegovu inspekciju
- „View Reports“ – pomoću koje je moguće pristupiti i analizirati izveštaje koji su generisani u procesu sinteze

U našem primeru pritisnućemo dugme *Cancel*, kako bi se vratili u glavni prozor Vivado Design Suite alata, čiji sadržaj bi trebao da odgovara onome sa slike 1.32.



Slika 1.32. Izgled glavnog prozora Vivado Design Suite alata nakon što je završena faza sinteze dizajna

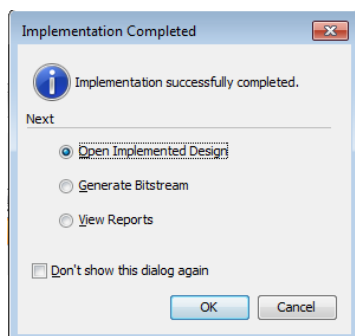
Sa slike 1.32 možemo videti da Vivado Design Suite alat pruža indikaciju o tome koji je proces poslednji završen, u našem slučaju to je proces sinteze. U gornjem desnom uglu možemo videti poruku „Synthesis Complete“, kao indikaciju da je proces sinteze završen. U okviru „Project Summary“ prozora takođe možemo videti indikaciju o tome da je proces sinteze uspešno završen (*Synthesis* tab, *Status* polje, *Complete*) i da prilikom procesa sinteze Vivado alat nije generisao nikakve poruke o upozorenjima ili greškama (*Synthesis* tab, *Messages* polje, „No error or warnings“). Ukoliko pogledamo status za proces implementacije možemo videti da ovaj proces još uvek nije bio

pokrenut (*Implementation* tab, *Status* polje, „Not started“). U okviru „*Project Summary*“ prozora takođe možemo videti i izveštaj o hardverskim resursima koji su neophodni da bi se sintetizovao dizajn (*Utilization* tab, „Post-Synthesis“).

Pored ovoga, unutar „*Tcl Console*“ kartice možemo videti koje su sve Tcl komande bile izvršene od strane Vivado alata u toku procesa sinteze. Takođe, unutar *Reports* kartice možemo videti da su se pojavili izveštaji nakon završene sinteze („*Vivado Synthesis Report*“ i „*Utilization Report*“). Dvoklikom na neki od ova dva izveštaja, moguće je analizirati tok sinteze i analizirati angažovane hardverske resurse. Ovaj korak je vrlo važan, jer analizom izveštaja o sintezi dizajner može da se uveri kako je alat za sintezu razumeo i sintetizovao razvijeni VHDL model, kao i da li je u procesu sinteze došlo do nekih problema ili grešaka. Detaljna analiza izveštaja nakon sinteze, kao i ostalih izveštaja, izlazi iz okvira ovog kursa.

Nakon završene sinteze dizajna, ukoliko smo zadovoljni načinom na koji je VHDL model sistema preveden u *gate-level* netlistu, možemo pristupiti sledećem koraku, fazi implementacije dizajna. Kao što je ranije već rečeno, ova faza uključuje tri zasebna koraka: translaciju, mapiranje i raspoređivanje i povezivanje. Klikom na komandu „*Run Implementation*“, u okviru *Implementation* grupe, izvršavaju se navedena tri koraka. Kao što je ranije već bilo rečeno, u fazi translacije dolazi do spajanja više odvojenih *gate-level* netlisti u jednu, konačnu netlistu, koja će biti korišćena u narednim koracima implementacije. Prilikom implementacije, Vivado Design Suite alat će takođe voditi računa o prostornim i vremenskim ograničenjima koja su navedena u *and_gate.xdc* fajlu, i pokušaće da dizajn rasporedi po FPGA kolu na takav način da ova ograničenja budu zadovoljena.

Kao i ranije, nakon završetka faze implementacije, Vivado Design Suite alat će o tome obavestiti korisnika, otvaranjem „*Implementation Completed*“ prozora, prikazanog na slici 1.33, i kreirati odgovarajuće fajlove sa izveštajima. U našem primeru izgled glavnog prozora Vivado Design Suite alata nakon završene faze implementacije trebalo bi da odgovara onom sa slike 1.33.

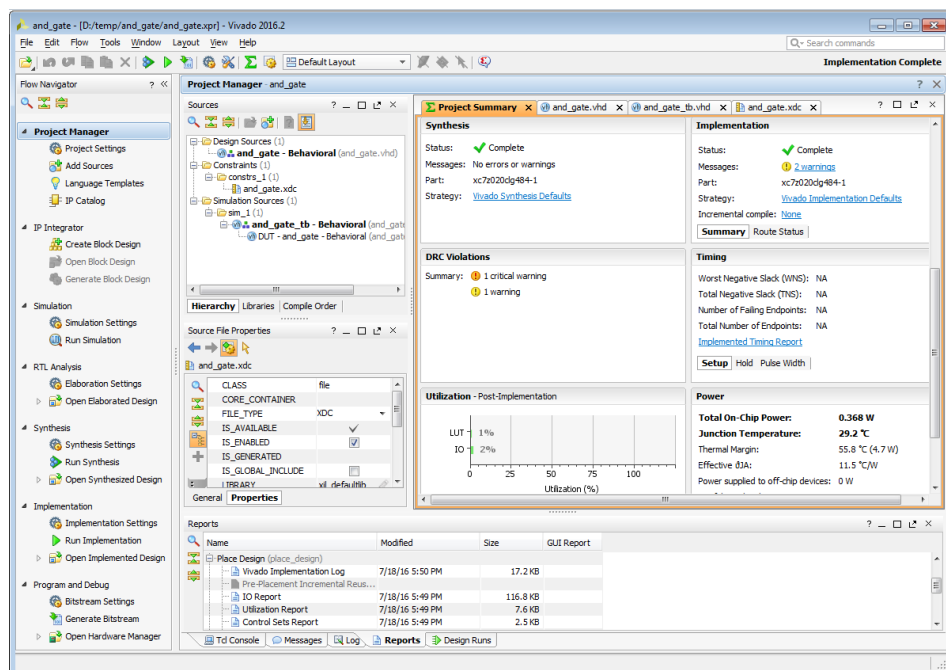


Slika 1.33. „*Implementation Completed*“ prozor koji se pojavljuje nakon što je završena faza implementacije dizajna

„Implementation Completed“ prozor nudi tri akcije:

- „Open Implemented Design“ – pomoću koje je moguće otvoriti implementirani dizajn i izvršiti njegovu inspekciju
- „Generate Bitstream“ – pomoću koje je moguće pokrenuti proces generisanja konfiguracionog fajla
- „View Reports“ – pomoću koje je moguće pristupiti i analizirati izveštaje koji su generisani u procesu implementacije

U našem primeru pritisnućemo dugme *Cancel*, kako bi se vratili u glavni prozor Vivado Design Suite alata, čiji sadržaj bi trebao da odgovara onome sa slike 1.34.



Slika 1.34. Izgled glavnog prozora Vivado Design Suite alata nakon što je završena faza implementacije dizajna

Sa slike 1.34 možemo videti da je sada i *Implementation* proces kompletiran, da se rezultati u pogledu potrebnih hardverskih resursa sada odnose na one nakon završene faze implementacije (*Utilization – Post-Implementation*), kao i da su se pojavili novi izveštaji u okviru *Reports* kartice.

Ukoliko smo zadovoljni načinom na koji je dizajn implementarian, poslednji korak prilikom realizacije digitalnog sistema pomoću FPGA komponenti je faza generisanja

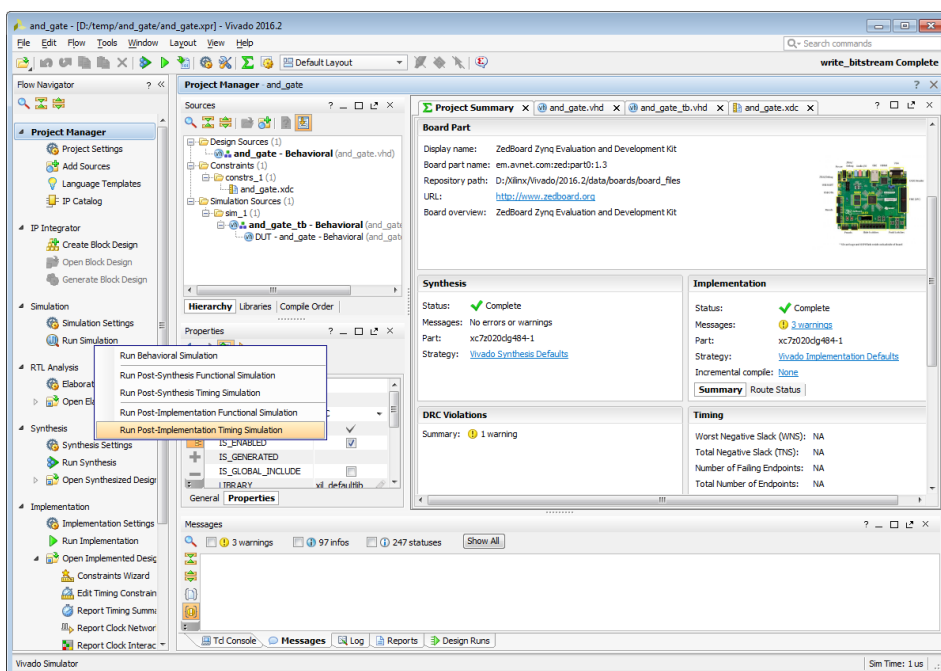
konfiguracionog fajla. Ona se u Vivado Design Suite alatu pokreće pritiskom na dugme „Generate Bitstream“, u okviru „Program and Debug“ grupe akcija.

1.7. Vremenska simulacija i statička vremenska analiza

Kao što je ranije bilo rečeno, nakon završetka faze raspoređivanja i povezivanja, imamo kompletne informacije o kašnjenjima svakog od unutrašnjih signala koji su prisutni u sistemu. Koristeći ove informacije moguće je sprovesti i drugi tip simulacije, o kojem je bilo reči u uvodnom delu ove glave, vremensku simulaciju. Takođe, korišćenjem ovih podataka, moguće je sprovesti i statičku vremensku analizu.

Vremenska simulacija

Prvi korak u izvođenju vremenske simulacije je da se generiše vremenski tačan model implementiranog dizajna. Ovaj model Vivado alat generiše automatski, na osnovu izbora tipa simulacije koju korisnik želi da sprovede. Pritiskom na dugme „Run Simulation“, u okviru *Simulation* grupe, pojavljuje se dodatni meni iz kojega korisnik može izabrati koji tip simulacije želi da bude izvršen, kao što se vidi na slici 1.35.



Slika 1.35. Moguće vrste simulacije koje stoje na raspolaganju nakon završene faze implementacije dizajna

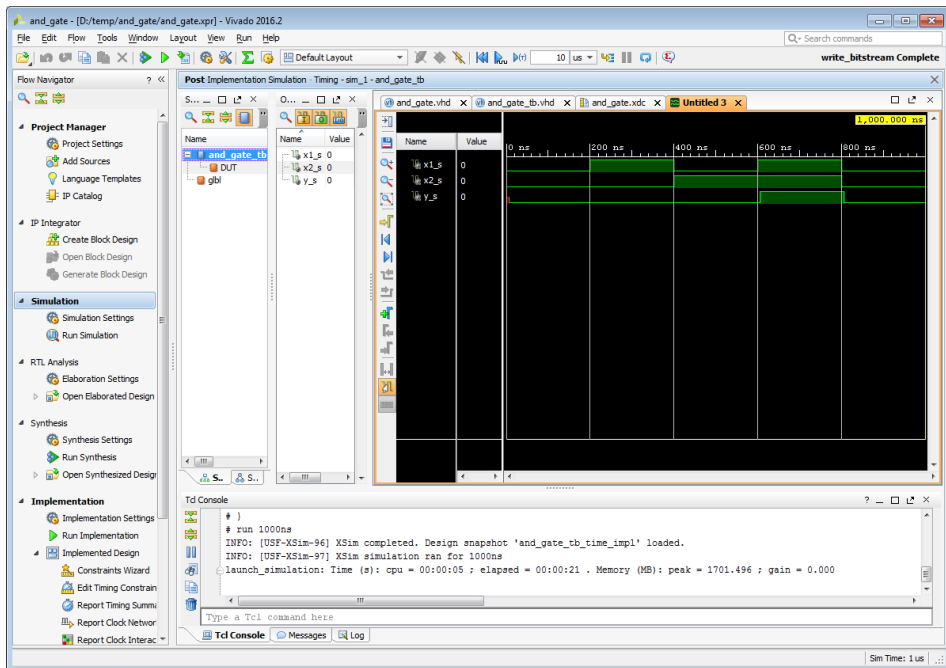
Vivado alat u stanju je da izvrši 5 različitih vrsta simulacije:

- **Funkcionalna simulacija HDL modela dizajna** – „*Run Behavioral Simulation*“ opcija – ovo je zapravo simulacija VHDL modela dizajna pri čemu se smatra da su kašnjenja svih signala unutar dizajna jednaka nuli o kojoj je bilo reči ranije.
- **Funkcionalna simulacija sintetizovanog dizajna** – „*Run Post-Synthesis Functional Simulation*“ – ovo je funkcionalna simulacija automatski generisanog HDL modela dizajna nakon što je izvršena sinteza dizajna. Vivado Design Suite alat automatski generiše simulacioni HDL model koristeći sintetizovanu netlistu, i ovaj model se zatim koristi u simulaciji. Svrha ove simulacije je da potvrdi da prilikom sinteze dizajna nije došlo do unošenja nekih grešaka koje bi bile posledica prisustva bagova u samom alatu za sintezu.
- **Vremenska simulacija sintetizovanog dizajna** – „*Run Post-Synthesis Timing Simulation*“ – ovo je vremenska simulacija automatski generisanog HDL modela dizajna nakon što je izvršena sinteza dizajna. Za razliku od funkcionalne simulacije sintetizovanog dizajna, u ovom slučaju kašnjenja unutrašnjih signala nisu jednaka nuli već se ona estimiraju koristeći odgovarajuće statističke procese. Ova estimacija naravno odstupa od stvarnih kašnjenja, koja će biti poznata tek nakon implementacije, i po pravilu predstavlja najbolji slučaj, tako da se može iskoristiti za procenu očekivanog ponašanja dizajna i pre njegove implementacije. Ukoliko se utvrdi da sintetizovani dizajn sa pretpostavljenim kašnjenjima unutrašnjih signala ne zadovoljava uslove specifikacije, postupak realizacije može se prekinuti već u ovoj fazi.
- **Funkcionalna simulacija implementiranog dizajna** – „*Run Post-Implementation Functional Simulation*“ – ovo je funkcionalna simulacija automatski generisanog HDL modela dizajna nakon što je izvršena implementacija dizajna. Vivado Design Suite alat automatski generiše simulacioni model koristeći raspoređen i povezan dizajn, i ovaj model se zatim koristi u simulaciji. Svrha ove simulacije je da potvrdi da prilikom implementacije dizajna nije došlo do unošenja nekih grešaka koje bi bile posledica prisustva bagova u alatima za transliranje, mapiranje, raspoređivanje i povezivanje.
- **Vremenska simulacija implementiranog dizajna** – „*Run Post-Implementation Timing Simulation*“ – ovo je vremenska simulacija automatski generisanog HDL modela dizajna nakon što je izvršena implementacija dizajna. Ovo je najtačnija simulacija projektovanog sistema i ona u potpunosti odgovara ponašanju sistema kada se on realizuje na nekoj FPGA komponenti. Kako je sada poznat tačan raspored svih hardverskih modula na FPGA komponenti i poznate su dužine svih unutrašnjih veza, moguće je vrlo precizno odrediti kašnjenja svakog od unutrašnjih signala, što na kraju doprinosi vrlo tačnim rezultatima vremenske simulacije.

NAPOMENA: Obratite pažnju da tačnost simulacije raste kako se krećemo na dole duž gornje liste. Najnepreciznija je funkcionalna simulacija HDL modela dizajna, dok je najtačnija vremenska simulacija implementiranog dizajna. Međutim, vreme trajanja simulacije takođe se povećava kako se krećemo u istom smeru. Najbrža simulacija je funkcionalna simulacija HDL modela dizajna, dok je najsporija vremenska simulacija implementiranog dizajna.

NAPOMENA: Obratite pažnju da se za potrebe svake od navedenih simulacija koristi **isti** VHDL testbenč fajl koji je bio korišćen za bihevioralnu simulaciju! Na ovaj način nije neophodno razvijanje različitih testbenč fajlova za različite tipove simulacije. Ovo je potpuno korektno, jer su testovi koje želimo da izvršimo nad bihevioralnim i vremenski tačnim modelima sistema identični, i nema razloga da pišemo različite testbenč fajlove.

U našem slučaju od ponuđenih opcija potrebno je izabrati poslednju, „*Run Post-Implementation Timing Simulation*“, jer želimo da prilikom simulacije koristimo automatski generisani vremenski tačan simulacioni model, nakon što je završena faza implementacije dizajna. Nakon završetka vremenske simulacije, tokom specificiranog intervala vremena (podrazumevana vrednost intervala vremena je 1000 ns, isto kao i u slučaju bihevioralne simulacije), otvara se prozor Vivado simulatora, prikazan na slici 1.36.



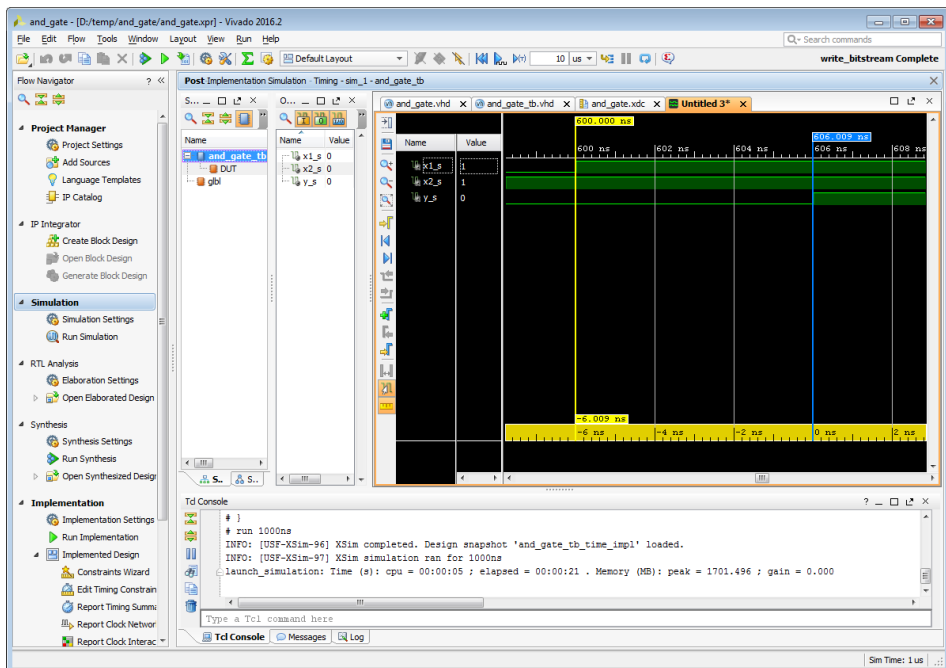
Slika 1.36. Izgled Vivado simulator prozora nakon završene simulacije vremenski tačnog simulacionog modela dvoulaznog I logičkog kola automatski generisanog nakon završene faze implementacije

Na prvi pogled, čini se da ne postoji razlika u talasnim oblicima prikazanim na slici 1.36 i onim sa slike 1.26, koja prikazuje rezultate simulacije bihevioralnog modela dvoulaznog I logičkog kola. Međutim, ukoliko se pažljivije pogledaju talasni oblici izlaznih signala y_s , može se uočiti da postoji jedna bitna razlika. U trenutku kada ulazni signali $x1_s$ i $x2_s$ menjaju vrednost sa logičke nule na logičku jedinicu (trenutak 600 ns na slici 1.36), izlazni signal y_s ne menja svoju vrednost sa logičke nule na jedinicu trenutno, već nakon nekog intervala vremena. Na slici 1.26 možemo videti da je izlazni signal y_s trenutno promenio svoju vrednost. Ovo je upravo suštinska razlika između funkcionalne i vremenske simulacije. Prilikom funkcionalne simulacije nisu nam poznata stvarna kašnjenja signala unutar modela, te ih zanemarujemo i podrazumevamo da se svi signali prostiru sa nultim kašnjenjem i da se sve promene na ulazima blokova trenutno manifestuju na njihovim izlazima. Zbog toga izlazni signal y trenutno menja svoju vrednost sa logičke nule na logičku jedinicu, u istom trenutku kada se desi odgovarajuća promena na ulazima dvoulaznog I logičkog kola. U slučaju vremenske simulacije, poznata su stvarna kašnjenja svakog unutrašnjeg signala (ili su ona estimirana sa većom ili manjom tačnošću), te se sada svi signali prostiru kroz sistem konačnom brzinom i potrebno je izvesno vreme da se promene na ulazima blokova propagiraju do njihovih izlaza. Ovo je realno ponašanje sistema, koje odgovara onome

što će se stvarno dešavati „u hardveru“, kada se dizajn fizički implementira pomoću odgovarajuće FPGA ili ASIC komponente.

NAPOMENA: Konačna i različita kašnjenja unutrašnjih signala mogu dovesti do pojave gličeva na izlaznim signalima kombinacionih mreža. Ukoliko se ovi signali dovode na ulaze lečeva, pojave gličeva mogu uzrokovati nepravilni rad čitavog sistema. Kako prilikom funkcionalne simulacije svi signali putuju beskonačno brzo, nije moguća pojava gličeva i sistem će se ponašati prema očekivanjima. Ovo je razlog zbog kojega je neophodno i bitno izvršiti i vremensku simulaciju pre fabrikacije digitalnog sistema, pogotovo ukoliko se u sistemu koriste lečevi.

Pomoću Vivado simulatora možemo i izmeriti koliko ovo kašnjenje iznosi. Ukoliko zumiramo region oko simulacionog vremena od 600 ns (korišćenjem opcije „Zoom In“, postavimo postojeći kursor na trenutak 600 ns i dodamo jedan marker (korišćenjem opcije „Markers/Add Marker“, koja se pojavljuje pritiskom na desno dugme miša, dok se miš nalazi u prozoru sa talasnim oblicima) dobijamo informaciju o vremenu propagacije promene na ulazima I logičkog kola do njegovog izlaza, kao što je prikazano na slici 1.37.



Slika 1.37. Propagaciono kašnjenje signala kroz dvoulazno I logičko kolo nakon završene faze implementacije dizajna

Sa slike 1.37 možemo videti da propagaciono kašnjenje kroz sistem, nakon završene faze implementacije dizajna, iznosi 6.009 ns.

Statička vremenska analiza

Statička vremenska (tajming) analiza na analitički i formalan način izračunava kašnjenja signala duž svakog od mogućih unutrašnjih puteva u dizajnu i proverava da li kritične putanje (putanje sa najvećim kašnjenjem) zadovoljavaju postavljena vremenska ograničenja (specificirana u fajlu sa ograničenjima), odnosno da li implementirani dizajn može da radi na zahtevanoj učestanosti klok signala, ukoliko u sistemu postoji klok signal. Statička tajming analiza je vrlo brza i zahteva vrlo malo računskog vremena da bi se kompletirala.

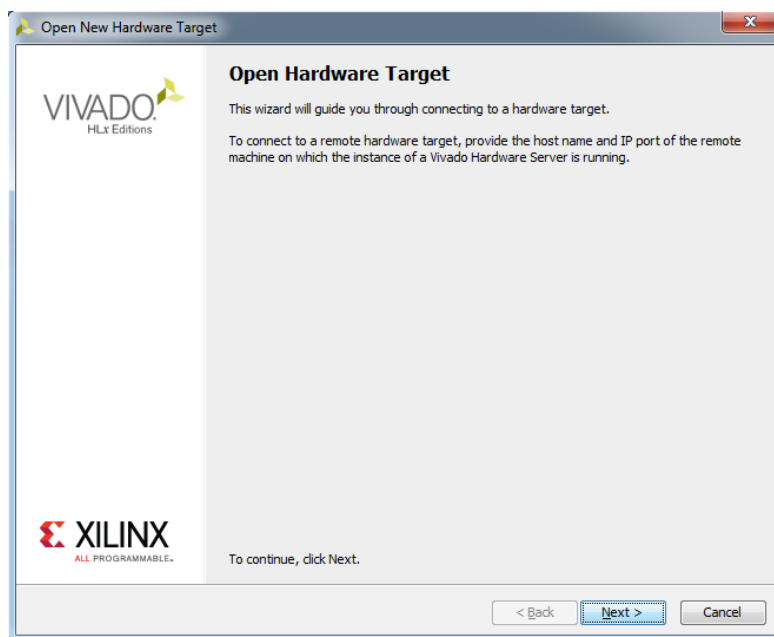
NAPOMENA: Pomoću statičke tajming analize ne može se utvrditi da li je dizajn funkcionalno korektan, niti da li u njemu postoje gličevi, niti da li će postojanje gličeva uticati na korektno ponašanje sistema. Za odgovore na ova pitanja mora se koristiti vremenska simulacija. Sa druge strane, ukoliko se digitalni sistem projektuje primenom metodologije sinhronog sekvencijalnog projektovanja, onda je korišćenje vremenske simulacije (koja je vrlo „skupa“ u pogledu vremena potrebnog za njeno kompletiranje) suvišno. Dovoljno je samo analizirati izveštaj nakon kompletiranja statičke vremenske analize i, ukoliko je on pozitivan (u smislu da su vremenska kašnjenja signala duž kritičnih putanja kraća od zahtevanih), može se sa sigurnošću garantovati da će projektovani sistem imati očekivano ponašanje ukoliko bude radio na brzinama koje su manje ili jednake brzinama koje su bile specificirane u fajlu sa ograničenjima.

Pomoću Vivado Design Suite alata moguće je sprovesti i statičku vremensku analizu. U okviru *Implementation* grupe potrebno je pokrenuti komandu „*Open Implemented Design*“. Nakon što Vivado alat pređe u režim analize implementiranog dizajna, u okviru „*Implemented Design*“ grupe potrebno je pokrenuti „*Report Timing Summary*“ akciju. Više o načinu sprovođenja statičke vremenske analize, kao i o načinu razumevanja generisanih izveštaja od strane Vivado Design Suite alata biće reči u poslednjoj, devetoj glavi ovog udžbenika.

1.8. Programiranje FPGA komponente i testiranje

Nakon što smo implementirali dizajn, možemo pristupiti njegovom testiranju na konkretnom hardveru, u našem slučaju na ZedBoard razvojnom sistemu. FPGA komponenta koja je prisutna na razvojnom sistemu može se i isprogramirati pomoću Vivado Design Suite alata, koristeći „*Open Target*“ akciju, unutar „*Open Hardware Manager*“ komande, koja se nalazi u okviru „*Program and Debug*“ grupe.

Pritiskom na „*Open Target*“ akciju pojavljuje se novi meni iz kojega je potrebno izabrati opciju „*Open New Target...*“, nakon čega se otvara „*Open New Hardware Target*“ prozor, prikazan na slici 1.38.

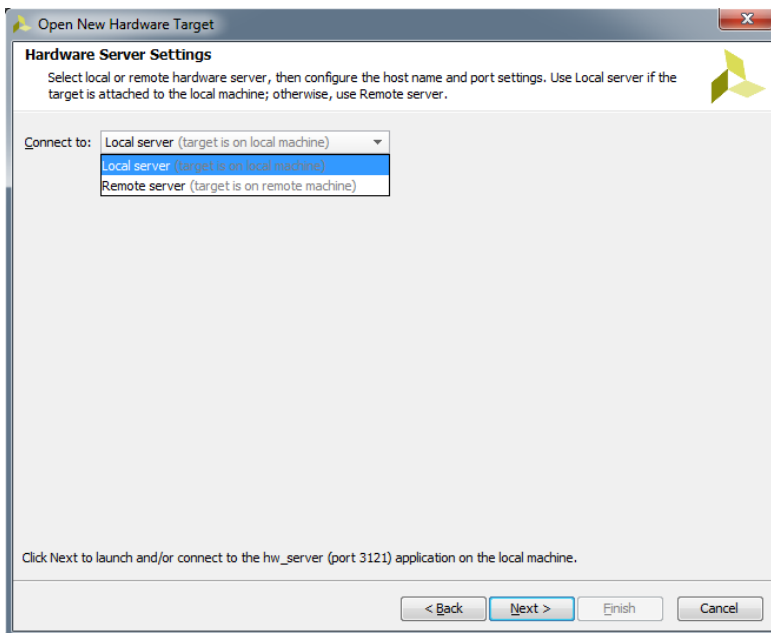


Slika 1.38. Izgled „*Open New Hardware Target*“ prozora

Pritiskom na dugme *Next* prelazimo na „*Hardware Server Settings*“ prozor, prikazan na slici 1.39. U okviru ovog prozora potrebno je izabrati hardverski server preko kojega će se pristupiti FPGA razvojnom sistemu. Vivado Design Suite alat pruža dve mogućnosti:

- pristup korišćenjem udaljenog servera (*Remote Server*), pomoću kojega je moguće pristupiti FPGA razvojnom sistemu koji se nalazi povezan na neki udaljeni računar koji je povezan na mrežu,
- pristup korišćenjem lokalnog servera (*Local Server*), pomoću kojega se pristupa FPGA razvojnom sistemu koji je povezan na lokalni računar, na kojem se izvršava Vivado Design Suite alat.

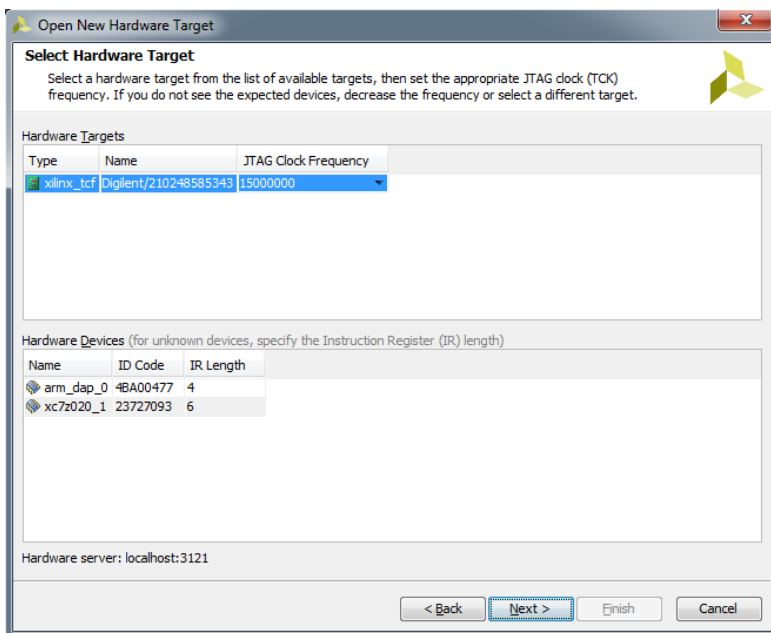
U našem slučaju potrebno je odabrati „*Local Server*“ opciju, kao što je prikazano na slici 1.39.



Slika 1.39. „Hardware Server Settings“ prozor sa potrebnim podešavanjima

Pritiskom na dugme *Next* Vivado Design Suite alat ostvaruje vezu sa FPGA razvojnim sistemom, koji je povezan na lokalni računar i utvrđuje koje se sve FPGA komponente nalaze na FPGA razvojnom sistemu. Nakon što se završi postupak identifikacije menja se sadržaj „Hardware Server Settings“ prozora i ispisuju se sve dostupne FPGA komponente. U našem primeru novi izgled „Hardware Server Settings“ prozora prikazan je na slici 1.40.

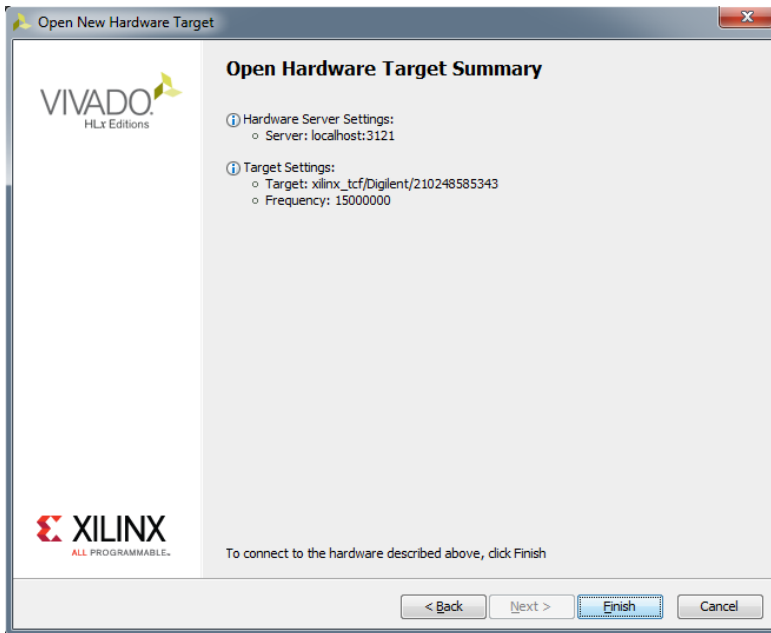
Sa slike 1.40 možemo videti da se na ZedBoard razvojnom sistemu nalazi jedna FPGA komponenta, *xc7z20_1*. Pored nje, Vivado Design Suite alat identifikao je još jedan uređaj, ARM procesirajući sistem *arm_dap_0*, koji je takođe prisutan u okviru *xc7z20* FPGA komponente, jer je reč o Zynq FPGA familiji kompanije Xilinx, koja pored programabilne zone sadrži i dvojezgarni ARM procesor.



Slika 1.40. Izgled „Hardware Server Settings“ prozora nakon završenog procesa identifikacije FPGA komponenti

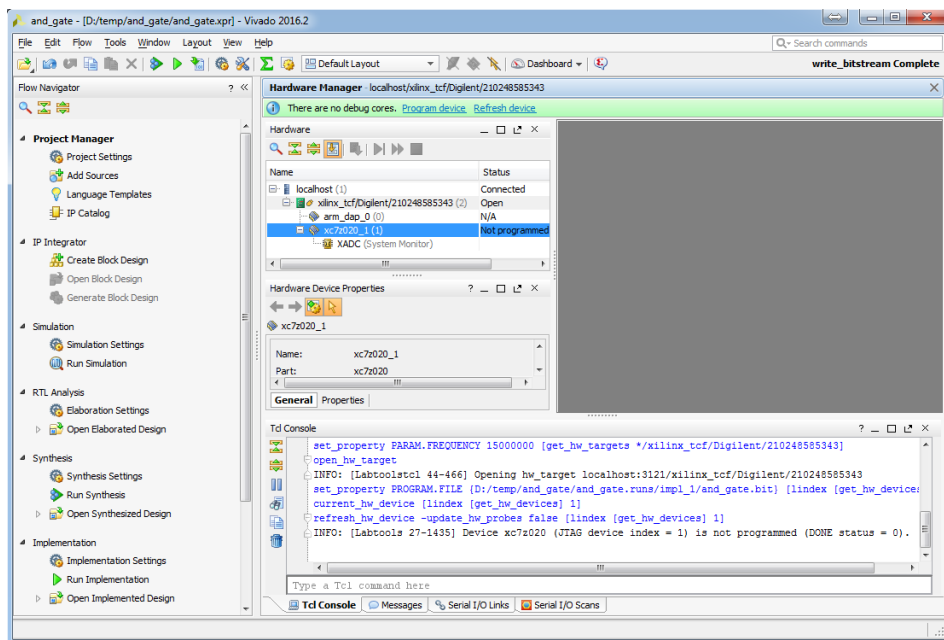
Pritiskom na dugme *Next* završava se proces ostvarivanja konekcije sa odabranim FPGA razvojnim sistemom, a Vivado Design Suite alat prikazuje završni „Open Hardware Target Summary“ prozor, prikazan na slici 1.41.

U okviru ovog prozora prikazane su karakteristike odabranog hardverskog servera koji je korišćen za ostvarivanje veze sa FPGA razvojnim sistemom (*Hardware Server Settings*), kao i karakteristike uspostavljene veze (*Target Settings*).



Slika 1.41. Izgled „Open Hardware Target Summary“ prozora nakon završenog procesa povezivanja sa FPGA razvojnim sistemom

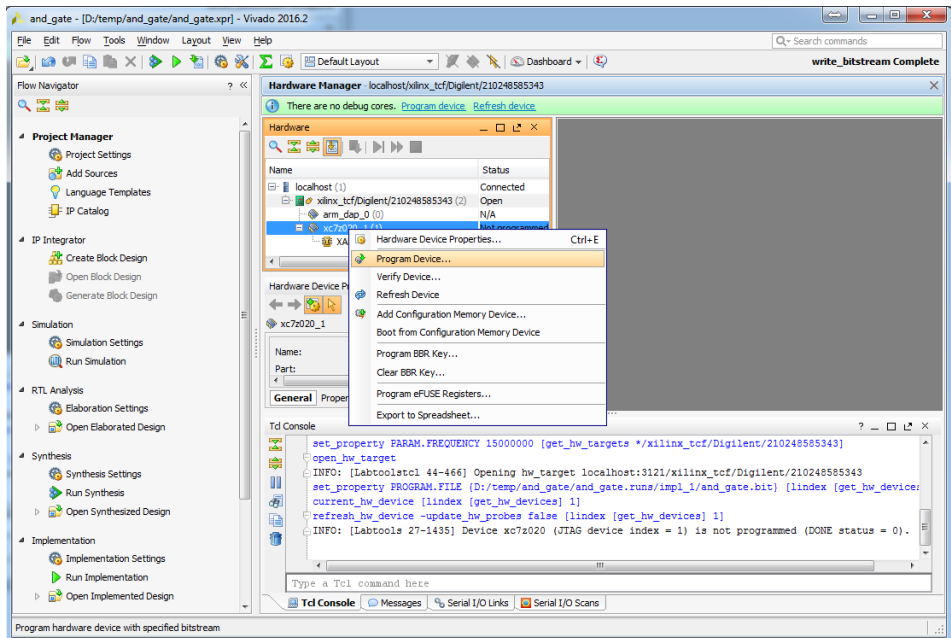
Pritiskom na dugme Next vraćamo se u osnovni prozor Vivado Design Suite alata, u kome se sada pojavljuje novi prozor, „*Hardware Manager*“, prikazan na slici 1.42. U okviru „*Hardware Manager*“ obavljaju se sve radnje vezane za programiranje odabrane FPGA komponente koja postoji na FPGA razvojnom sistemu sa kojim je ostvarena konekcija, kao i eventualno upravljanje integrisanim logičkim analizatorom (*ILA Core*) i virtuelnim ulazno/izlaznim modulom (*VIO Core*), ukoliko su oni prisutni u dizajnu koji je trenutno implementiran pomoću odbrane FPGA komponente. Ova dva modula su vrlo korisni u procesu hardverskog testiranja implementiranog dizajna, ali način njihovog korišćenja izlazi iz okvira ovog kursa.



Slika 1.42. Početni izgled „Hardware Manager“ prozora u okviru glavnog Vivado Design Suite prozora, nakon ostvarene konekcije sa FPGA razvojnim sistemom

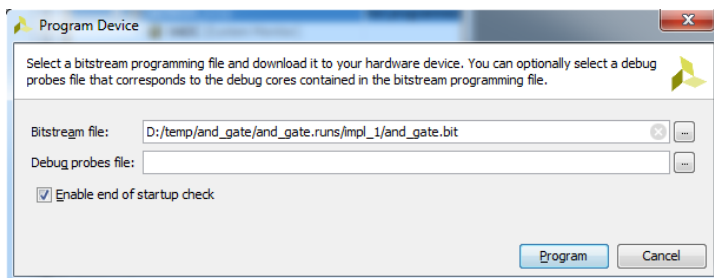
Sa slike 1.42 možemo videti da su dostupni hardverski resursi, koji su prisutni na ZedBoard razvojnom sistemu, ARM procesorski sistem (*arm_dap_0* modul u okviru *Hardware* prozora) i FPGA programabilna logika (*xc7z20_1* modul u okviru *Hardware* prozora). Vidimo da FPGA programabilna logika još uvek nije konfigurisana (polje u okviru *Status* kolone pridruženo *xc7z20_1* modulu ima sadržaj „Not programmed“). Ovo je očekivano, jer mi tek treba da isprogramiramo FPGA komponentu sa konfiguracionim fajlom dvoulaznom logičkog I kola, koji smo generisali na kraju poglavlja 1.6.

Proces programiranja FPGA komponente sa odabranim konfiguracionim fajlom započinje selekcijom *xc7z20_1* modula u okviru *Hardware* prozora i pritiskom na desno dugme miša, nakon čega se pojavljuje novi prozor prikazan na slici 1.43. U okviru ovog prozora nalaze se sve akcije koje je moguće izvršiti nad odabranom FPGA komponentom. Kako mi želimo da programiramo odabranu FPGA komponentu sa novim konfiguracionim fajlom, potrebno je odabrati akciju „Program Device...“, kao što je prikazano na slici 1.43.



Slika 1.43. Lista akcija koje je moguće izvršiti nad xc7z20_1 FPGA komponentom unutar „Hardware Manager“ prozora

Nakon aktiviranja „Program Device...“ akcije pojavljuje se „Program Device“ prozor, prikazan na slici 1.44.

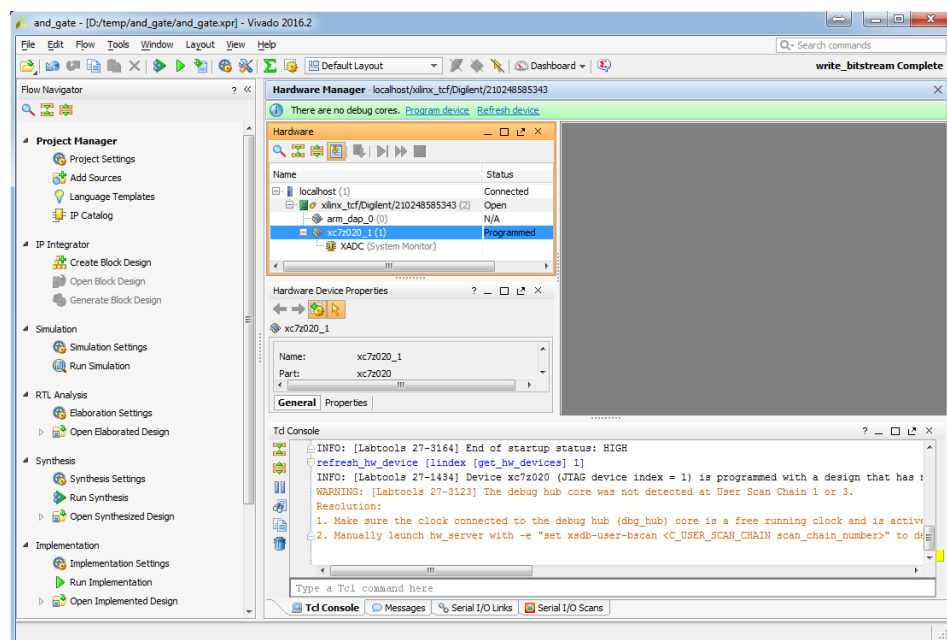


Slika 1.44. „Program Device“ prozor sa potrebnim podešavanjima

U okviru „Program Device“ prozora potrebno je odabrati konfiguracioni fajl sa kojim želimo da programiramo odabranu FPGA komponentu („Bitstream file“ polje) i opcionalno fajl koji sadrži opis sonde za debugovanje, u slučaju da unutar dizajna koji će biti implementiran pomoću FPGA komponente postoje ILA ili VIO moduli („Debug probes

file“ polje). U našem slučaju potrebna podešavanja treba da imaju izgled prikazan na slici 1.44.

Pritiskom na dugme *Program* vrši se samo programiranje FPGA komponente sa odabranim konfiguracionim fajlom. Nakon završetka procesa programiranja, Vivado Design Suite se vraća nazad u „*Hardware Manager*“ prozor, u kome je status FPGA komponente koja je upravo programirana (u našem primeru to je *xc7z20_1* FPGA komponenta) promenjen i sada glasi *Programmed*, kao što se može videti na slici 1.45.



Slika 1.45. Izgled „*Hardware Manger*“ prozora nakon programiranja FPGA komponente

Ovim je proces programiranja odabrane FPGA komponente završen i ona sada implementira željeni digitalni sistem, u našem slučaju dvoulazno I logičko kolo.

Rad dvoulaznog I logičkog kola sada možemo testirati i na konkretnom hardveru, ZedBoard razvojnom sistemu. Promenom položaja *SW0* i *SW1* prekidača i observacijom stanja *LD0* LED diode moguće je verifikovati ispravan rad dvoulaznog I logičkog kola.

